Tasca S2.01

Nivell 1

Exercici 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Es revisen els arxius i s'observa que l'arxiu "N1-Ex.1__ estructura dades.sql" conté l'estructura d'un esquema de nom transactions, composta per dues taules, la taula company i la taula transaction.

En l'arxiu "N1-Ex.1__dades_introduir.sql" s'observa que conté les dades pròpiament dites, a inserir tant a la taula company com a la taula transaction.

Es creen les dues taules:

```sql
CREATE DATABASE IF NOT EXISTS transactions;
USE transactions;

CREATE TABLE IF NOT EXISTS company (
    id VARCHAR(15) PRIMARY KEY,
    company_name VARCHAR(255),
    phone VARCHAR(15),
    email VARCHAR(100),
    country VARCHAR(100),
    website VARCHAR(255)
);

CREATE TABLE IF NOT EXISTS transaction (
    id VARCHAR(255) PRIMARY KEY,
    credit_card_id VARCHAR(15) REFERENCES credit_card(id),
    company_id VARCHAR(20),
    user_id INT REFERENCES user(id),
    lat FLOAT,
    longitude FLOAT,
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN,
    FOREIGN KEY (company_id) REFERENCES company(id)
);
```

```
[2026-04-21 15:33:00] Connected
[2026-04-21 15:33:00] > use transactions
[2026-04-21 15:33:00] completed in 3 ms
[2026-04-21 15:40:54] transactions> CREATE DATABASE IF NOT EXISTS transactions
[2026-04-21 15:40:54] 1 row affected in 15 ms
[2026-04-21 15:40:54] transactions> USE transactions
[2026-04-21 15:40:54] completed in 6 ms
[2026-04-21 15:40:54] transactions> CREATE TABLE IF NOT EXISTS company (
                                        id VARCHAR(15) PRIMARY KEY,
                                        company_name VARCHAR(255),
                                        phone VARCHAR(15),
                                        email VARCHAR(100),
                                        country VARCHAR(100),
                                        website VARCHAR(255)
                                    )
[2026-04-21 15:40:54] completed in 45 ms
[2026-04-21 15:40:54] transactions> CREATE TABLE IF NOT EXISTS transaction (
                                        id VARCHAR(255) PRIMARY KEY,
                                        credit_card_id VARCHAR(15) REFERENCES credit_card(id),
                                        company_id VARCHAR(20),
                                        user_id INT REFERENCES user(id),
                                        lat FLOAT,
                                        longitude FLOAT,
                                        timestamp TIMESTAMP,
                                        amount DECIMAL(10, 2),
                                        declined BOOLEAN,
                                        FOREIGN KEY (company_id) REFERENCES company(id)
                                    )
[2026-04-21 15:40:54] completed in 50 ms
```

Després es carreguen les dades, utilitzant la comanda Run de l'editor, ja que l'intent de carregar les dades a un arxiu en blanc sql per executar-lo ens provoca el bloqueig de l'ordinador.

Run 'N1-Ex.1_dades_intro...' Ctrl+Shift+F10

```
[2026-04-21 16:12:29] 1000 row(s) affected in 3 sec, 628 ms
[2026-04-21 16:12:33] 1000 row(s) affected in 3 sec, 477 ms
[2026-04-21 16:12:36] 1000 row(s) affected in 3 sec, 518 ms
[2026-04-21 16:12:40] 1000 row(s) affected in 3 sec, 705 ms
[2026-04-21 16:12:44] 1000 row(s) affected in 3 sec, 699 ms
[2026-04-21 16:12:48] 1000 row(s) affected in 3 sec, 809 ms
[2026-04-21 16:12:52] 1000 row(s) affected in 3 sec, 350 ms
[2026-04-21 16:12:55] 1000 row(s) affected in 3 sec, 421 ms
[2026-04-21 16:12:59] 1000 row(s) affected in 3 sec, 727 ms
[2026-04-21 16:13:03] 1000 row(s) affected in 3 sec, 372 ms
```

```
[2026-04-21 16:17:10] Summary: 100,101 of 100,101 statements executed in 6 min, 24 sec, 642 ms (26,846,396 symbols in file)
```

L'esquema, com hem dit anteriorment, es diu **transactions**, i conté dues taules, **company** i **transaction**.

Es validen les dades de la taula **company**:

```
SHOW COLUMNS FROM company;
```

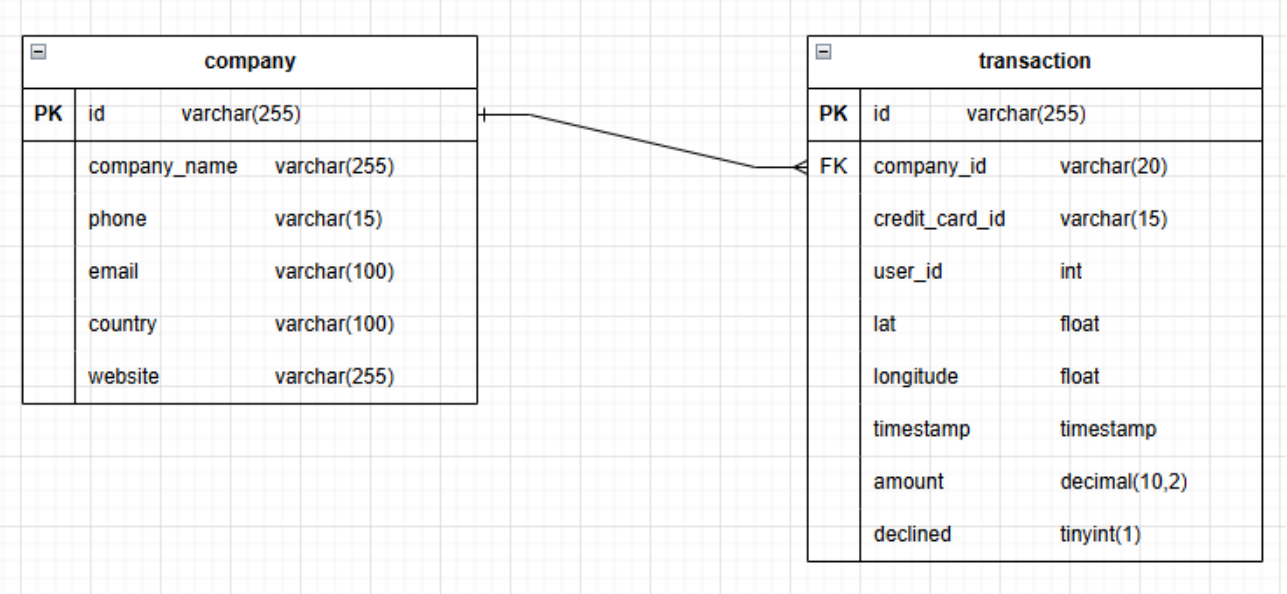
	Field	Type	Null	Key	Default	Extra
1	id	varchar(15)	NO	PRI	<null>	
2	company_name	varchar(255)	YES		<null>	
3	phone	varchar(15)	YES		<null>	
4	email	varchar(100)	YES		<null>	
5	country	varchar(100)	YES		<null>	
6	website	varchar(255)	YES		<null>	

I també es validen les dades de la taula **transaction:**

```
SHOW COLUMNS FROM transaction;
```

	Field	Type	Null	Key	Default	Extra
1	id	varchar(255)	NO	PRI	<null>	
2	credit_card_id	varchar(15)	YES		<null>	
3	company_id	varchar(20)	YES	MUL	<null>	
4	user_id	int	YES		<null>	
5	lat	float	YES		<null>	
6	longitude	float	YES		<null>	
7	timestamp	timestamp	YES		<null>	
8	amount	decimal(10,2)	YES		<null>	
9	declined	tinyint(1)	YES		<null>	

L'esquema un cop importats els arxius seria aquest:



Després de revisar-ho, s'haurien de fer els següents canvis:

Per millorar la llegibilitat, mantenir una coherència en la nomenclatura i evitar confusions, anomenar les columnes id de les taules **company** i **transaction** per company_id i transaction_id respectivament.

En la taula **company** s'hauria de modificar:

Camp company_id a varchar(10).
Camp company_name a NOT NULL.
Camp phone a varchar(20).
Es valora proposar posar camp email o phone a NOT NULL per que així sempre hi hagi una via de contacte amb la companyia.

En la taula **transaction** s'hauria de modificar:

Camp transaction_id a varchar(36).
Camp company_id a varchar(10) i NOT NULL.
Camp lat a decimal(11,8).
Camp longitude a decimal(11,8).
Camp amount a NOT NULL.
Camp timestamp a NOT NULL.
Camp user_id a NOT NULL.

La resta de camps els deixaria tal i com estan actualment.

Per modificar el nom del camp id de la taula **company**, primer s'ha de "trencar" la relació amb el camp id de la taula **transaction**, trobant primer la FK en la taula **transaction**:

```
SHOW CREATE TABLE transaction;
```

1 row

	Table	Create Table

1	transaction	CREATE TABLE `transaction` (
		  `id` varchar(255) NOT NULL,
		  `credit_card_id` varchar(15) DEFAULT NULL,
		  `company_id` varchar(20) DEFAULT NULL,
		  `user_id` int DEFAULT NULL,
		  `lat` float DEFAULT NULL,
		  `longitude` float DEFAULT NULL,
		  `timestamp` timestamp NULL DEFAULT NULL,
		  `amount` decimal(10,2) DEFAULT NULL,
		  `declined` tinyint(1) DEFAULT NULL,
		  PRIMARY KEY (`id`),
		  KEY `company_id` (`company_id`),
		  CONSTRAINT `transaction_ibfk_1` FOREIGN KEY (`company_id`) REFERENCES `company` (`id`)
		) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```
[2026-04-21 18:08:15] transactions> SHOW CREATE TABLE transaction
[2026-04-21 18:08:15] 1 row retrieved starting from 1 in 401 ms (execution: 14 ms, fetching: 387 ms)
```

Un cop trobada la FK, procedirem a eliminar-la:

```
ALTER TABLE transaction DROP FOREIGN KEY transaction_ibfk_1;
```

```
[2026-04-21 18:12:40] transactions> ALTER TABLE transaction DROP FOREIGN KEY transaction_ibfk_1
[2026-04-21 18:12:40] completed in 33 ms
```

Un cop eliminada la relació, podrem fer les modificacions a les dues taules:

```
ALTER TABLE company
    CHANGE COLUMN id company_id varchar(10) NOT NULL,
    MODIFY COLUMN company_name varchar(255) NOT NULL,
    MODIFY COLUMN phone varchar(20);
```

```
[2026-04-21 18:14:22] transactions> ALTER TABLE company
                                    CHANGE COLUMN id company_id varchar(10) NOT NULL,
                                    MODIFY COLUMN company_name varchar(255) NOT NULL,
                                    MODIFY COLUMN phone varchar(20)
[2026-04-21 18:14:22] 100 rows affected in 90 ms
```

```
ALTER TABLE transaction
    CHANGE COLUMN id transaction_id varchar(36) NOT NULL,
    MODIFY COLUMN company_id varchar(10) NOT NULL,
    MODIFY COLUMN lat decimal(11,8),
    MODIFY COLUMN longitude decimal(11,8),
    MODIFY COLUMN amount decimal(10,2),
    MODIFY COLUMN timestamp timestamp NOT NULL,
    MODIFY COLUMN user_id int NOT NULL;
```

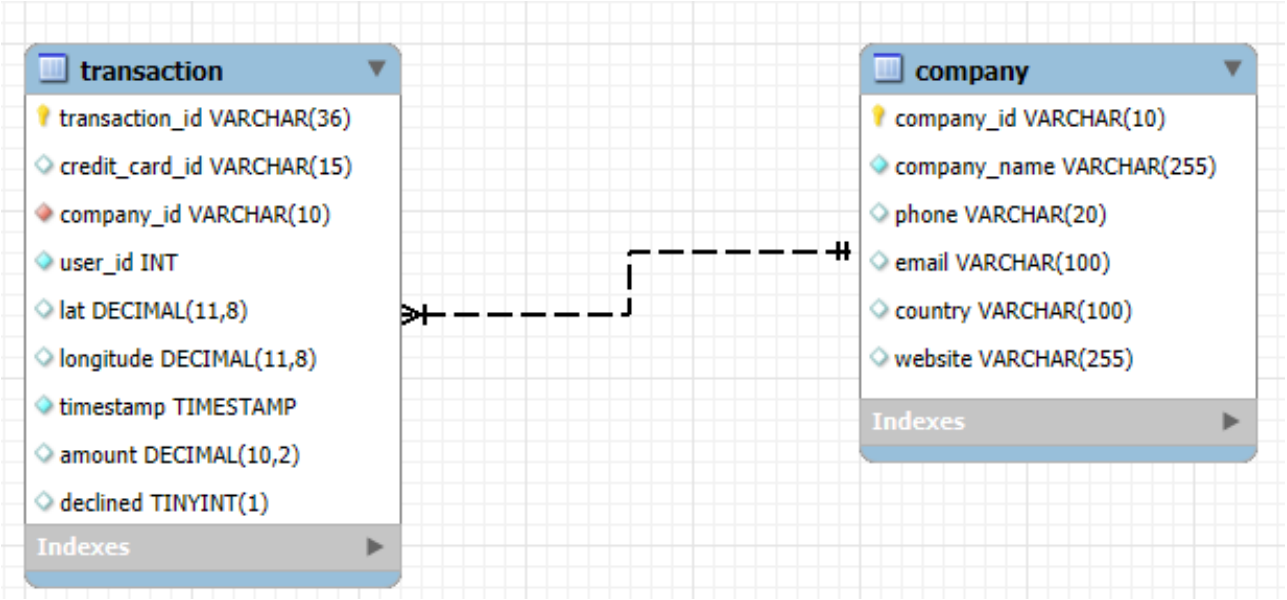
```
[2026-04-21 18:20:34] transactions> ALTER TABLE transaction
                                    CHANGE COLUMN id transaction_id varchar(36) NOT NULL,
                                    MODIFY COLUMN company_id varchar(10) NOT NULL,
                                    MODIFY COLUMN lat decimal(11,8),
                                    MODIFY COLUMN longitude decimal(11,8),
                                    MODIFY COLUMN amount decimal(10,2),
                                    MODIFY COLUMN timestamp timestamp NOT NULL,
                                    MODIFY COLUMN user_id int NOT NULL
[2026-04-21 18:20:36] [01000][1265] Data truncated for column 'lat' at row 1
[2026-04-21 18:20:36] [01000][1265] Data truncated for column 'longitude' at row 1
[2026-04-21 18:20:36] 100,000 rows affected in 2 s 34 ms
```

Un cop fet els canvis, tornem a generar la relació entre els camps company_id de les dues taules:

```
ALTER TABLE transaction
ADD CONSTRAINT fk_transaction_company
FOREIGN KEY (company_id) REFERENCES company (company_id);
```

```
[2026-04-21 18:26:02] transactions> ALTER TABLE transaction
                                    ADD CONSTRAINT fk_transaction_company
                                    FOREIGN KEY (company_id) REFERENCES company (company_id)
[2026-04-21 18:26:03] 100,000 rows affected in 1 s 548 ms
```

Un cop fets tots els canvis, el diagrama seria aquest:



Bàsicament, aquest diagrama ens indica que és un esquema EER, on hi ha una relació entre les dues taules a través del camp company_id, on a la taula **transaction** és Foreign Key (FK) i a la taula **company** és Primary Key (PK), de molts (**transaction**) a un (**company**).

Exercici 2

Utilitzant JOIN realitzaràs les següents consultes:

- Llistat dels països que estan generant vendes.

Es fa un SELECT DISTINCT per evitar que surtin països duplicats. Es fa un JOIN de la taula **transaction** per poder validar els països que han realitzat operacions.

```
SELECT DISTINCT c.country AS Països
FROM company AS c
JOIN transaction AS t ON c.company_id = t.company_id
WHERE t.declined = 0 AND c.country IS NOT NULL;
```

Output | Llistat dels països ...stan generant vendes.

15 rows | Tx: Auto | DDL | CSV

	Països
1	Germany
2	Australia
3	United States
4	New Zealand
5	Norway
6	United Kingdom
7	Italy
8	Belgium
9	Sweden
10	Ireland
11	China
12	Canada
13	France
14	Netherlands
15	Spain

Output | Llistat dels països ...stan generant vendes.

```
[2026-04-22 20:45:37] transactions> SELECT DISTINCT c.country AS Països
                                    FROM company AS c
                                    JOIN transaction AS t ON c.company_id = t.company_id
                                    WHERE t.declined = 0 AND c.country IS NOT NULL
[2026-04-22 20:45:37] 15 rows retrieved starting from 1 in 546 ms (execution: 224 ms, fetching: 322 ms)
```

- Des de quants països es generen les vendes.

Es fa un SELECT COUNT DISTINCT per contar els països però que no compti els duplicats. Es fa un JOIN de la taula **transaction** per poder validar els països que han realitzat operacions.

```
SELECT COUNT(DISTINCT c.country) AS Total_Països
FROM company AS c
JOIN transaction AS t ON c.company_id =t.company_id
WHERE t.declined = 0;
```

Output | Des de quants països...s generen les vendes.

1 row | CSV

	Total_Països
1	15

Output | Des de quants països...s generen les vendes.

```
[2026-04-22 22:03:59] transactions> SELECT COUNT(DISTINCT c.country) AS Total_Països
                                    FROM company AS c
                                    JOIN transaction AS t ON c.company_id =t.company_id
                                    WHERE t.declined = 0
[2026-04-22 22:04:00] 1 row retrieved starting from 1 in 971 ms (execution: 556 ms, fetching: 415 ms)
```

- Identifica la companyia amb la mitjana més gran de vendes.

Es calcula la mitjana i es fa un ROUND amb dos decimals per normalitzar. S'agrupa per l'id i el nom de la companyia i s'ordena pel càlcul de la quantitat.

```
SELECT c.company_id AS IDCompanyia, c.company_name AS Companyia, ROUND(AVG(t.amount),2) AS Mitjana_vendes
FROM company AS c
JOIN transaction AS t ON c.company_id =t.company_id
WHERE t.declined = 0
GROUP BY c.company_id, c.company_name
ORDER BY ROUND(AVG(t.amount),2) DESC
LIMIT 1;
```

Output | Identifica la compan...a més gran de vendes.

1 row | CSV

	IDCompanyia	Companyia	Mitjana_vendes
1	b-2222	Ac Fermentum Incorporated	284.91

Output | Identifica la compan...a més gran de vendes.

```
[2026-04-22 22:07:24] transactions> SELECT c.company_id AS IDCompanyia, c.company_name AS Companyia, ROUND(AVG(t.amount),2) AS
 Mitjana_vendes
                                     FROM company AS c
                                     JOIN transaction AS t ON c.company_id =t.company_id
                                     WHERE t.declined = 0
                                     GROUP BY c.company_id, c.company_name
                                     ORDER BY ROUND(AVG(t.amount),2) DESC
                                     LIMIT 1
[2026-04-22 22:07:26] 1 row retrieved starting from 1 in 1 s 131 ms (execution: 774 ms, fetching: 357 ms)
```

Exercici 3

Utilitzant només subconsultes (sense utilitzar JOIN):

- Mostra totes les transaccions realitzades per empreses d'Alemanya.

Es fa una selecció de les transaccions, filtrant pels company_id que son d'Alemanya.

```
SELECT t.transaction_id AS Transaccions
FROM transaction AS t
WHERE EXISTS (SELECT c.company_id
              FROM company AS c
              WHERE t.company_id = c.company_id AND c.country = 'Germany') AND t.declined = 0;
```

Output | Mostra totes les tra... empreses d'Alemanya.

13.269 rows | Tx: Auto | DDL | CSV

	Transaccions
1	00138D3B-206D-4C03-94B7-63A2676EB9B4
2	0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9
3	00201A11-2E62-44C4-941D-198FC8DB77F0
4	00235618-0A5C-4D49-9DCB-B3A9405D8923
5	005A5A7B-1F1A-4B6C-9B15-1625A78C9C38
6	00687139-48B2-4FFA-8E73-B20376F04AB4
7	0074F4DD-32F1-4827-8758-55896314623A

Output | Mostra totes les tra... empreses d'Alemanya.

```
[2026-04-22 22:12:22] transactions> SELECT t.transaction_id AS Transaccions
                                    FROM transaction AS t
                                    WHERE EXISTS (SELECT c.company_id
                                                  FROM company AS c
                                                  WHERE t.company_id = c.company_id AND c.country = 'Germany') AND t.declined = 0
[2026-04-22 22:12:22] 13,269 rows retrieved starting from 1 in 523 ms (execution: 19 ms, fetching: 504 ms)
```

- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.

Es selecciona el id de la companyia i el seu nom per evitar possibles confusions en el cas que el nom de l'empresa estigués duplicat. Es filtra a partir d'una subconsulta on s'ha calculat la mitjana de l'amount amb AVG i ROUND amb dos decimals.

```
SELECT c.company_id AS IDCompanyia, c.company_name AS Companyia
FROM company AS c
WHERE EXISTS (SELECT t.company_id
                FROM transaction AS t
                WHERE t.company_id = c.company_id AND t.declined = 0 AND t.amount > (SELECT ROUND(AVG(t.amount), 2)
                                                                                     FROM transaction AS t
                                                                                     WHERE t.declined = 0));
```

Output | Llista les empreses ...tes les transaccions.

100 rows | Tx: Auto | DDL | CSV

	IDCompanyia	Companyia
1	b-2222	Ac Fermentum Incorporated
2	b-2226	Magna A Neque Industries
3	b-2230	Fusce Corp.
4	b-2234	Convallis In Incorporated
5	b-2238	Ante Iaculis Nec Foundation
6	b-2242	Donec Ltd
7	b-2246	Sed Nunc Ltd
8	b-2250	Amet Nulla Donec Corporation

Output | Llista les empreses ...tes les transaccions.

```
[2026-04-22 22:16:38] transactions> SELECT c.company_id AS IDCompanyia, c.company_name AS Companyia
                                    FROM company AS c
                                    WHERE EXISTS (SELECT t.company_id
                                                  FROM transaction AS t
                                                  WHERE t.company_id = c.company_id AND t.declined = 0 AND t.amount >
                                                   (SELECT ROUND(AVG(t.amount), 2)

 FROM transaction AS t

 WHERE t.declined = 0))
[2026-04-22 22:16:38] 100 rows retrieved starting from 1 in 575 ms (execution: 214 ms, fetching: 361 ms)
```

- Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

Es selecciona el id de la companyia i el seu nom per evitar possibles confusions en el cas que el nom de l'empresa estigués duplicat. Es filtra a partir d'una subconsulta on s'ha calculat els id d'empresa que tenen transaccions, indicant que mostri les que no coincideixin amb la subconsulta.

```
SELECT c.company_id AS IDCompanyia, c.company_name AS Companyia
FROM company AS c
WHERE NOT EXISTS (SELECT t.company_id
                  FROM transaction AS t
                  WHERE c.company_id = t.company_id);
```

Output | Eliminaran del siste... d'aquestes empreses.

0 rows | Tx: Auto | DDL | CSV

IDCompanyia	Companyia

Output | Eliminaran del siste... d'aquestes empreses.

```
[2026-04-22 10:56:20] transactions> SELECT c.company_id AS IDCompanyia, c.company_name AS Companyia
                                    FROM company AS c
                                    WHERE c.company_id NOT IN (SELECT t.company_id
                                                               FROM transaction AS t)
[2026-04-22 10:56:21] 0 rows retrieved in 348 ms (execution: 11 ms, fetching: 337 ms)
```

Exercici 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

Primer revisem l'arxiu "N1-Ex.4__ datos_introducir_credit.sql". Observem els camps i els seus valors:

```
-- Insertamos datos de credit_card
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR301950312213576817638661', '5424465566813633', '3257', '984', '10/30/22');
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2945', 'DO26854763748537475216568689', '5142423821948828', '9080', '887', '08/24/23');
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2952', 'BG45IVQL52710525608255', '4556 453 55 5287', '4598', '438', '06/29/21');
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2959', 'CR7242477244335841535', '372461377349375', '3583', '667', '02/24/23');
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2966', 'BG72LKTQ15627628377363', '448566 886747 7265', '4900', '130', '10/29/24');
```

Després d'analitzar-ho, crearem tots els camps amb valor varchar, ja que encara que alguns dels camps només siguin camps numèrics, en realitat son números identificatius, i si enlloc de varchar fos int, el número 034 passaria a ser el 34.

El camp id un cop s'insereixin les dades canviarem el nom a **credit_card_id** per mantenir la nomenclatura, tal i com hem fet abans amb els camps id de **transaction** i **company**. Aquest camp tindrà un valor varchar(15) per poder realitzar després la relació amb el camp credit_card_id de la taula **transaction**.

El camp expiring_date té els valors 'mm/dd/aa', que és incorrecte. Primer inserirem com a varchar i després modificarem el valor amb STR_TO_DATE per poder passar-lo a date.

Un cop fet totes les modificacions, generarem la relació entre el camp credit_card_id de la taula **credit_card** amb el camp credit_card_id de la taula **transaction**.

Procedim a crear la taula:

```sql
USE transactions;
CREATE TABLE IF NOT EXISTS credit_card (
    id varchar(15) PRIMARY KEY,
    iban varchar(34),
    pan varchar(20),
    pin varchar(4),
    cvv varchar(3),
    expiring_date varchar(10)
);
```

```
[2026-04-22 11:02:05] transactions> CREATE TABLE IF NOT EXISTS credit_card (
                                        id varchar(15) PRIMARY KEY,
                                        iban varchar(34),
                                        pan varchar(20),
                                        pin varchar(4),
                                        cvv varchar(3),
                                        expiring_date varchar(10)
                                    )
[2026-04-22 11:02:05] completed in 62 ms
```

Validem els camps:

```sql
SHOW COLUMNS FROM credit_card;
```

Output | Result 21

6 rows | CSV

	Field	Type	Null	Key	Default	Extra
1	id	varchar(15)	NO	PRI	<null>	
2	iban	varchar(34)	YES		<null>	
3	pan	varchar(20)	YES		<null>	
4	pin	varchar(4)	YES		<null>	
5	cvv	varchar(3)	YES		<null>	
6	expiring_date	varchar(10)	YES		<null>	

```
Output    Result 21
[2026-04-22 11:04:49] transactions> SHOW COLUMNS FROM credit_card
[2026-04-22 11:04:50] 6 rows retrieved starting from 1 in 377 ms (execution: 17 ms, fetching: 360 ms)
```

Ara procedirem a inserir les dades des de l'arxiu N1-Ex.4__datos_introducir_credit.sql, però abans modificarem l'arxiu amb un "USE transactions;" perquè l'aplicació sàpiga a quin esquema s'han d'inserir les dades:

```
USE transactions;
-- Insertamos datos de credit_card
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR301950312213576817638661', '5424465566813633', '3257', '984', '10/30/22');
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2945', 'DO26854763748537475216568689', '5142423821948828', '9080', '887', '08/24/23'
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2952', 'BG45IVQL52710525608255', '4556 453 55 5287', '4598', '438', '06/29/21');
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2959', 'CR7242477244335841535', '372461377349375', '3583', '667', '02/24/23');
```

Fet això, executarem el fitxer:

Run 'N1-Ex.4__ datos_intr...'

```
[2026-04-22 11:19:27] Run C:\Users\USUARIO\Documents\ITADataAnalysis\Data\N1-Ex.4__ datos_introducir_credit.sql
USE transactions
[2026-04-22 11:19:28] 0 row(s) affected in 4 ms
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR301950312213576817638661', '5424465566813633', '3257', '984', '10/30/22')
.....................................................................................
[2026-04-22 11:19:32] 1000 row(s) affected in 3 sec, 799 ms
.....................................................................................
[2026-04-22 11:19:36] 1000 row(s) affected in 3 sec, 690 ms
.....................................................................................
[2026-04-22 11:19:39] 1000 row(s) affected in 3 sec, 556 ms
.....................................................................................
[2026-04-22 11:19:43] 1000 row(s) affected in 3 sec, 509 ms
.....................................................................................
[2026-04-22 11:19:47] 1000 row(s) affected in 3 sec, 575 ms
[2026-04-22 11:19:47] Summary: 5,001 of 5,001 statements executed in 19 sec, 509 ms (803,736 symbols in file)
```

Un cop tenim les dades introduïdes, realitzarem els canvis proposats:

Primer, canviarem els valors per poder assignar DATE al camp expiring_date i així poder mantenir la integritat:

```
UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y');
ALTER TABLE credit_card MODIFY expiring_date DATE;
```

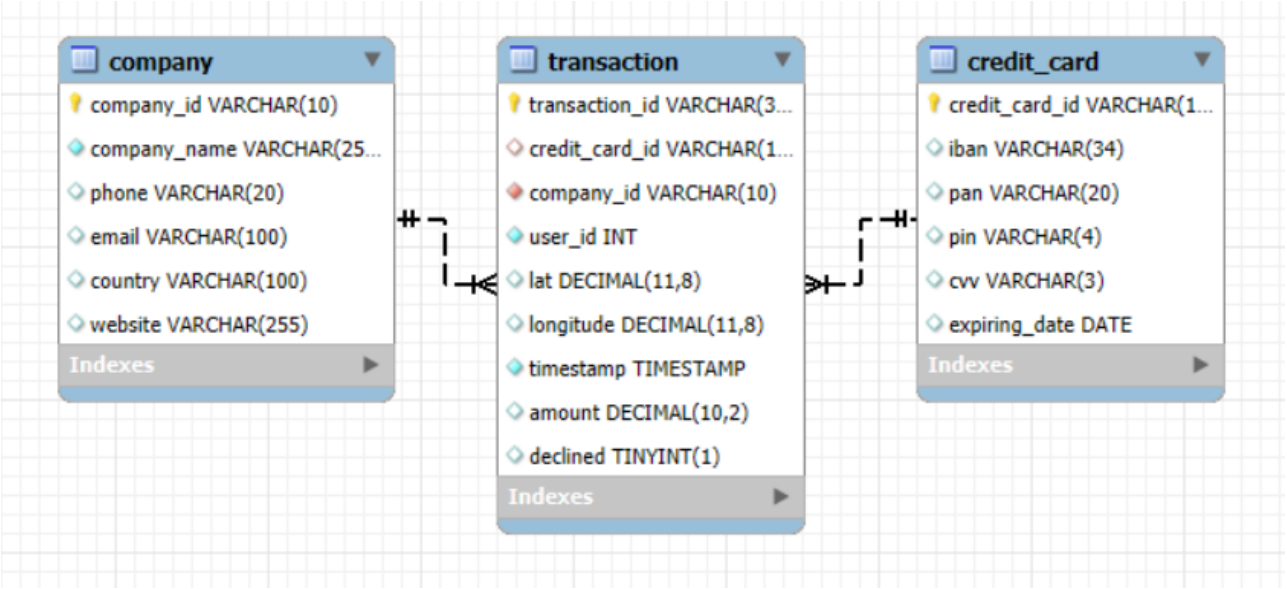
Un cop fets aquests canvis, modificarem el nom del camp id:

```
ALTER TABLE credit_card CHANGE COLUMN id credit_card_id varchar(15);
```

Posteriorment, generarem la relació entre els camps credit_card_id de la taula **transaction** i la taula **credit_card**, alhora que convertim credit_card_id de la taula **transaction** en FK.

```
ALTER TABLE transaction
ADD CONSTRAINT fk_transaction_creditcard
FOREIGN KEY (credit_card_id) REFERENCES credit_card (credit_card_id);
```

Un cop realitzats tots els canvis, el nou diagrama de l'esquema transactions quedaria així:



Després dels canvis realitzats, ara aquest esquema és un model d'estrella, on la taula **transaction** seria la taula de fets i les taules **company** i **credit_card** serien les taules de dimensions.

La relació entre **transaction** i **company** continua sent de molts a un a través dels camps company_id de cada taula i la relació entre **transaction** i **credit_card** també és de molts a un a través dels camps credit_card_id.

Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

Fem un UPDATE, filtrant pel id de la targeta:

```
UPDATE credit_card
SET iban = 'TR323456312213576817699999'
WHERE credit_card.credit_card_id = 'CcU-2938';
```

```
Output    transactions.credit_card
[2026-04-22 11:52:54] 100,000 rows affected in 2 s 154 ms
[2026-04-22 12:37:55] transactions> UPDATE credit_card
                                    SET iban = 'TR323456312213576817699999'
                                    WHERE credit_card.credit_card_id = 'CcU-2938'
[2026-04-22 12:37:55] 1 row affected in 14 ms
```

Per validar que s'ha realitzat el canvi, executem el següent codi:

```
SELECT iban
FROM credit_card
WHERE credit_card.credit_card_id = 'CcU-2938';
```

```
Output    transactions.credit_card
1 row    Tx: Auto    DDL    CSV
| iban                |
| 1 TR323456312213576817699999 |

Output    transactions.credit_card
[2026-04-22 12:39:44] transactions> SELECT iban
                                    FROM credit_card
                                    WHERE credit_card.credit_card_id = 'CcU-2938'
[2026-04-22 12:39:44] 1 row retrieved starting from 1 in 335 ms (execution: 4 ms, fetching: 331 ms)
```

Exercici 6

En la taula "transaction" ingressa una nova transacció amb la següent informació:

Id
108B1D1D-5B23-A76C-55EF-C568E49A99DD

credit_card_id
CcU-9999

company_id
b-9999

user_id
9999

lat
829.999

longitude
-117.999

amount
111.11

declined
0

Com que estem en un esquema relacional, abans de poder inserir les dades a la taula **transaction** s'ha d'inserir el company_id i company_name a la taula **company** i també s'ha d'inserir la credit_card_id a la taula **credit_card**.

Inserim el id de la targeta:

```
INSERT INTO credit_card (credit_card_id)
VALUES ( credit_card_id 'CcU-9999');
```

Output | transactions.credit_card

```
[2026-04-22 13:16:51] transactions> INSERT INTO credit_card (credit_card_id)
                                    VALUES ('CcU-9999')
[2026-04-22 13:16:51] 1 row affected in 10 ms
```

I ara inserim el id de la companyia i el seu nom (després de consultar-ho amb RRHH, ens proporcionen el nom de l'empresa):

```
INSERT INTO company (company_id, company_name)
VALUES ( company_id 'b-9999', company_name 'Inventa Turba');
```

Output | transactions.credit_card

```
[2026-04-22 13:33:21] transactions> INSERT INTO company (company_id, company_name)
                                    VALUES ('b-9999', 'Inventa Turba')
[2026-04-22 13:33:21] 1 row affected in 12 ms
```

I ara ja podem inserir les dades a la taula **transaction**:

```
INSERT INTO transaction
VALUES ( transaction_id '108B1D1D-5B23-A76C-55EF-C568E49A99DD', credit_card_id 'CcU-9999', company_id 'b-9999', user_id 9999,
 lat '829.999', longitude '-117.999', timestamp NOW(), amount 111.11, declined 0);
```

Output | transactions.credit_card

```
[2026-04-22 13:36:57] transactions> INSERT INTO transaction
                                    VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, '829.999',
                                     '-117.999', NOW(), 111.11, 0)
[2026-04-22 13:36:57] 1 row affected in 12 ms
```

Exercici 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

Procedim a eliminar la columna indicada amb DROP COLUMN:

```
ALTER TABLE credit_card
DROP COLUMN pan;
```

Output | transactions.credit_card

```
[2026-04-22 13:42:58] transactions> ALTER TABLE credit_card
                                    DROP COLUMN pan
[2026-04-22 13:42:58] completed in 49 ms
```

I validem que s'ha eliminat:

```
SHOW COLUMNS FROM credit_card;
```

Output | Result 42

5 rows | CSV

	Field	Type	Null	Key	Default	Extra
1	credit_card_id	varchar(15)	NO	PRI	<null>	
2	iban	varchar(34)	YES		<null>	
3	pin	varchar(4)	YES		<null>	
4	cvv	varchar(3)	YES		<null>	
5	expiring_date	date	YES		<null>	

Output | Result 42

```
[2026-04-22 13:44:54] transactions> SHOW COLUMNS FROM credit_card
[2026-04-22 13:44:54] 5 rows retrieved starting from 1 in 347 ms (execution: 7 ms, fetching: 340 ms)
```

Exercici 8

Descarrega els arxius CSV que **trobaràs a l'apartat de recursos**:

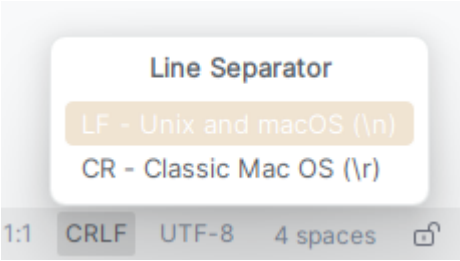
- american_users.csv
- european_users.csv
- companies.csv
- credit_cards.csv
- transactions.csv

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Primer de tot, obrim els arxius i revisem la codificació.
Tots els arxius tenen la codificació UTF-8, però 4 dels arxius tenen el final de línia Unix (LF) mentre que el cinquè té Windows(CRLF).

Windows (CRLF)	UTF-8
Unix (LF)	UTF-8

Normalitzem (igualem) els arxius, convertint l'arxiu amb Windows (CRLF) a Unix (LF):



Des de l’editor fem una simulació de importació de dades per veure com es carreguen.

Tots carreguen correctament:

Data Preview

file: N1.Ex.8__credit_cards.csv format: CSV

	id	user_id	iban	pan	pin	cvv	track1
1	CcU-2938	275	TR301950312213576817638661	5424465566813633	3257	984	%B8383712448554646^Wo
2	CcU-2945	274	DO26854763748537475216568689	5142423821948828	9080	887	%B4621311609958661^Uf
3	CcU-2952	273	BG45IVOL52710525608255	4556 453 55 5287	4598	438	%B2183285104307501^Cd

Data Preview

file: N1.Ex.8__companies.csv format: CSV

	company_id	company_name	phone	email	countr
1	b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germar
2	b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Austra
3	b-2230	Fusce Corp	08 14 97 58 85	risus@protonmail.edu	United

Data Preview

file: N1.Ex.8__european_users.csv format: CSV

	id	name	surname	phone	email	birth_date	country
1	151	Meghan	Hayden	0800 746 6747	arcu.vel@hotmail.ca	Jul 2, 1980	United Kingdom
2	152	Hakeem	Alford	(0111) 367 0184	adipiscing.ligula@google.edu	Sep 30, 1979	United Kingdom
3	153	Keegan	Pugh	(016977) 3851	sodales.nisi@aol.org	Jul 27, 1994	United Kingdom

Data Preview

file: N1-Ex.8__american_users.csv format: CSV

	id	name	surname	phone	email	birth_date	country
1	1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	Nov 17, 1985	United Sta
2	2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	Aug 23, 1992	United Sta
3	3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	Apr 29, 1998	United Sta

Data Preview

file: N1.Ex.8__transactions.csv format: Semicolon-separated

	id	card_id	business_id	timestamp	amount	declined	pr
1	CDDA7E40-544D-47BB-A4ED-671DD8A950D9	CcS-6894	b-2466	2018-12-12 08:05:17	161.88	0	75
2	09456357-8E9B-475A-8257-87A023699964	CcS-5135	b-2342	2024-05-20 22:49:08	171.13	0	3
3	C47C7C84-C174-4973-A76B-825A9DB5582A	CcS-8415	b-2250	2018-11-04 23:16:18	497.29	0	92

1. Arquitectura de l'Esquema
S'ha dissenyat l'esquema de nom financial_db com un model d’estrella relacional format per quatre taules: operations com a taula de fets i companies, credit_cards i users com a taules dimensionals.

2. Consolidació de l'entitat users
Un punt clau del disseny ha estat la integració dels fitxers american_users i european_users. Després de validar que ambdós conjunts de dades compartien una estructura idèntica (mateix nombre de columnes, noms i formats de dades), s’unificaran en una única taula **users**. Per mantenir la traçabilitat de l'origen, s’afegirà un camp addicional per categoritzar la regió (Amèrica o Europa).

3. Estratègia de Càrrega i Importació Segura
Per minimitzar riscos de truncament o errors de format durant la fase d'ingesta des dels fitxers .csv, s'ha optat per una càrrega en dues fases:

Primera fase: Creació de taules amb columnes de tipus VARCHAR(255) sense assignar PK ni FK. Els noms dels camps es mantenen fidels a les capçaleres dels fitxers originals per facilitar la càrrega durant l'ordre LOAD DATA.

Segona fase: Execució de la càrrega massiva de dades a cada taula.

4. Normalització, Neteja i Refactorització (Data Cleaning)
Un cop les dades s'hagin inserit a la base de dades (a les seves taules respectives), es procedirà a la seva transformació:

Validació: Comprovació del volum de registres i coherència de la càrrega.

Estandardització: Anomenat de columnes per unificar la nomenclatura (com ara canviar id per identificadors més descriptius com company_id o card_id).

Neteja: Aplicació de funcions de depuració per eliminar espais en blanc o caràcters especials que poguessin corrompre les dades.

Tipificació: Conversió dels camps VARCHAR als seus tipus de dades definitius (com DECIMAL per a imports, DATE per a dates i INT per algunes claus).

5. Definició d'Integritat Referencial
Com a pas final,s'assignaran les Primary Key (PK) per garantir la unicitat de cada registre i s'han configurat les Foreign Key (FK). Aquesta estructura final assegurarà que totes les transaccions de la taula **operations** estiguin correctament vinculades amb les seves respectives empreses, targetes i usuaris, blindant la consistència de la base de dades.

Procedim a crear l'esquema:

```
CREATE DATABASE if NOT EXISTS financial_db;
```

```
[2026-04-22 19:01:13] transactions> CREATE DATABASE if NOT EXISTS financial_db
[2026-04-22 19:01:13] 1 row affected in 15 ms
```

```
USE financial_db;
```

```
[2026-04-22 19:02:28] transactions> USE financial_db
[2026-04-22 19:02:28] completed in 3 ms
```

Ara procedim a crear les taules:

```
CREATE TABLE IF NOT EXISTS companies
(
    company_id varchar(255),
    company_name varchar(255),
    phone varchar(255),
    email varchar(255),
    country varchar(255),
    website varchar(255)
);
```

Output Eliminaran del siste... d'aquestes empreses.

```
[2026-04-22 23:13:07] financial_db> CREATE TABLE IF NOT EXISTS companies
                                    (
                                        company_id varchar(255),
                                        company_name varchar(255),
                                        phone varchar(255),
                                        email varchar(255),
                                        country varchar(255),
                                        website varchar(255)
                                    )
[2026-04-22 23:13:07] completed in 72 ms
```

```sql
CREATE TABLE IF NOT EXISTS operations
(
    id varchar(255),
    card_id varchar(255),
    business_id varchar(255),
    timestamp varchar(255),
    amount varchar(255),
    declined varchar(255),
    product_ids varchar(255),
    user_id varchar(255),
    lat varchar(255),
    longitude varchar(255)
);
```

Output | Eliminaran del siste... d'aquestes empreses.

```
[2026-04-22 23:18:19] financial_db> CREATE TABLE IF NOT EXISTS operations
                                    (
                                        id varchar(255),
                                        card_id varchar(255),
                                        business_id varchar(255),
                                        timestamp varchar(255),
                                        amount varchar(255),
                                        declined varchar(255),
                                        product_ids varchar(255),
                                        user_id varchar(255),
                                        lat varchar(255),
                                        longitude varchar(255)
                                    )
[2026-04-22 23:18:19] completed in 79 ms
```

```sql
CREATE TABLE IF NOT EXISTS credit_cards
(
    id varchar(255),
    user_id varchar(255),
    iban varchar(255),
    pan varchar(255),
    pin varchar(255),
    cvv varchar(255),
    track1 varchar(255),
    track2 varchar(255),
    expiring_date varchar(255)
);
```

Output | COUNT(company_id):int

```
[2026-04-23 00:42:41] financial_db> CREATE TABLE IF NOT EXISTS credit_cards
                                    (
                                        id varchar(255),
                                        user_id varchar(255),
                                        iban varchar(255),
                                        pan varchar(255),
                                        pin varchar(255),
                                        cvv varchar(255),
                                        track1 varchar(255),
                                        track2 varchar(255),
                                        expiring_date varchar(255)
                                    )
[2026-04-23 00:42:41] completed in 46 ms
```

```sql
CREATE TABLE IF NOT EXISTS users
(
    id varchar(255),
    name varchar(255),
    surname varchar(255),
    phone varchar(255),
    email varchar(255),
    birth_date varchar(255),
    country varchar(255),
    city varchar(255),
    postal_code varchar(255),
    address varchar(255),
    region varchar(255)
);
```

Output | Eliminaran del siste... d'aquestes empreses.

```
[2026-04-22 23:29:26] financial_db> CREATE TABLE IF NOT EXISTS users
                                    (
                                        id varchar(255),
                                        name varchar(255),
                                        surname varchar(255),
                                        phone varchar(255),
                                        email varchar(255),
                                        birth_date varchar(255),
                                        country varchar(255),
                                        city varchar(255),
                                        postal_code varchar(255),
                                        address varchar(255),
                                        region varchar(255)
                                    )
[2026-04-22 23:29:26] completed in 40 ms
```

Ara procedim a carregar les dades de american_users:

```sql
LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1-Ex.8__american_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
SET region = 'America';
```

Output | 👉 Necessitem conèix... venut cada producte.

```
[2026-04-22 23:48:19] financial_db> LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1-Ex.8__american_users
.csv'
                                    INTO TABLE users
                                    FIELDS TERMINATED BY ','
                                    ENCLOSED BY '"'
                                    LINES TERMINATED BY '\n'
                                    IGNORE 1 ROWS
                                    (id, name, surname, phone, email, birth_date, country, city, postal_code, address)
                                    SET region = 'America'
[2026-04-22 23:48:19] 1,010 rows affected in 100 ms
```

Validem que les dades s'han carregat correctament:

```sql
SELECT COUNT(users.id)
FROM users;
```

Output | COUNT(users.id):int

1 row | CSV

	`COUNT(users.id)`
1	1010

Output | COUNT(users.id):int

```
[2026-04-22 23:54:12] financial_db> SELECT COUNT(users.id)
                                    FROM users
[2026-04-22 23:54:12] 1 row retrieved starting from 1 in 480 ms (execution: 24 ms, fetching: 456 ms)
```

Carreguem més dades a **users**, des de european_users:

```sql
LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1.Ex.8__european_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
    ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
SET region = 'Europe';
```

Output | COUNT(users.id):int

```
[2026-04-23 00:22:24] financial_db> LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1.Ex.8__european_users
.csv'
                                    INTO TABLE users
                                    FIELDS TERMINATED BY ','
                                        ENCLOSED BY '"'
                                    LINES TERMINATED BY '\n'
                                    IGNORE 1 ROWS
                                    (id, name, surname, phone, email, birth_date, country, city, postal_code, address)
                                    SET region = 'Europe'
[2026-04-23 00:22:24] 3,990 rows affected in 116 ms
```

Validem que les dades s'han carregat correctament:

```sql
SELECT COUNT(users.id)
FROM users
WHERE region = 'Europe';
```

Output | COUNT(users.id):int

1 row | CSV

	`COUNT(users.id)`
1	3990

Output | COUNT(users.id):int

```
[2026-04-23 00:23:33] financial_db> SELECT COUNT(users.id)
                                    FROM users
                                    WHERE region = 'Europe'
[2026-04-23 00:23:33] 1 row retrieved starting from 1 in 403 ms (execution: 33 ms, fetching: 370 ms)
```

Seguim carregant dades, ara a la taula **companies**:

```sql
LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1.Ex.8__companies.csv'
INTO TABLE companies
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Output | COUNT(users.id):int

```
[2026-04-23 00:33:14] financial_db> LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1.Ex.8__companies.csv'
                                    INTO TABLE companies
                                    FIELDS TERMINATED BY ','
                                    LINES TERMINATED BY '\n'
                                    IGNORE 1 ROWS
[2026-04-23 00:33:14] 100 rows affected in 23 ms
```

Validem la càrrega de dades:

```sql
SELECT COUNT(company_id)
FROM companies;
```

Output | COUNT(company_id):int

1 row | CSV

	`COUNT(company_id)`
1	100

```
Output    COUNT(company_id):int
[2026-04-23 00:37:27] financial_db> SELECT COUNT(company_id)
                                    FROM companies
[2026-04-23 00:37:27] 1 row retrieved starting from 1 in 395 ms (execution: 7 ms, fetching: 388 ms)
```

Es carreguen les dades de **credit_cards**:

```
LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1.Ex.8__credit_cards.csv'
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
Output    COUNT(company_id):int
[2026-04-23 00:45:12] financial_db> LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1.Ex.8__credit_cards
.csv'
                                    INTO TABLE credit_cards
                                    FIELDS TERMINATED BY ','
                                    LINES TERMINATED BY '\n'
                                    IGNORE 1 ROWS
[2026-04-23 00:45:12] 5,000 rows affected in 272 ms
```

Es valida la càrrega:

```
SELECT COUNT(id)
FROM credit_cards;
```

Output | COUNT(id):int

1 row | CSV

	`COUNT(id)`
1	5000

```
Output    COUNT(id):int
[2026-04-23 00:48:20] financial_db> SELECT COUNT(id)
                                    FROM credit_cards
[2026-04-23 00:48:20] 1 row retrieved starting from 1 in 388 ms (execution: 13 ms, fetching: 375 ms)
```

Finalment, carreguem les dades a **operations**:

```
LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1.Ex.8__transactions.csv'
INTO TABLE operations
FIELDS TERMINATED BY ';'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
Output    COUNT(id):int
[2026-04-23 00:55:50] financial_db> LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N1.Ex.8__transactions
.csv'
                                    INTO TABLE operations
                                    FIELDS TERMINATED BY ';'
                                    LINES TERMINATED BY '\n'
                                    IGNORE 1 ROWS
[2026-04-23 00:55:52] 100,000 rows affected in 2 s 563 ms
```

Validem la càrrega:

```
SELECT COUNT(id)
FROM operations;
```

Output | COUNT(id):int

1 row | CSV

	`COUNT(id)`
1	100000

```
Output    COUNT(id):int
[2026-04-23 00:57:27] financial_db> SELECT COUNT(id)
                                    FROM operations
[2026-04-23 00:57:28] 1 row retrieved starting from 1 in 461 ms (execution: 128 ms, fetching: 333 ms)
```

Ara procedim a validar els valors de les dades, que no hi hagi id duplicats:

Primer ho fem amb la taula **users**:

```
SELECT COUNT(users.id)
FROM users;
```

Output | COUNT(users.id):int

1 row | CSV

`COUNT(users.id)`
5000

Output | COUNT(users.id):int

```
[2026-04-23 09:24:19] financial_db> SELECT COUNT(users.id)
                                    FROM users
[2026-04-23 09:24:19] 1 row retrieved starting from 1 in 370 ms (execution: 16 ms, fetching: 354 ms)
```

```
SELECT COUNT(DISTINCT id)
FROM users;
```

Output | COUNT(DISTINCT id):int

1 row | CSV

`COUNT(DISTINCT id)`
5000

Output | COUNT(DISTINCT id):int

```
[2026-04-23 09:26:06] financial_db> SELECT COUNT(DISTINCT id)
                                    FROM users
[2026-04-23 09:26:06] 1 row retrieved starting from 1 in 365 ms (execution: 21 ms, fetching: 344 ms)
```

Després amb la taula **companies**:

```
SELECT COUNT(company_id)
FROM companies;
```

Output | COUNT(company_id):int

1 row | CSV

`COUNT(company_id)`
100

Output | COUNT(company_id):int

```
[2026-04-23 00:37:27] financial_db> SELECT COUNT(company_id)
                                    FROM companies
[2026-04-23 00:37:27] 1 row retrieved starting from 1 in 395 ms (execution: 7 ms, fetching: 388 ms)
```

```
SELECT COUNT(DISTINCT company_id)
FROM companies;
```

Output | COUNT(DISTINCT company_id):int

1 row | CSV

`COUNT(DISTINCT company_id)`
100

Output | COUNT(DISTINCT company_id):int

```
[2026-04-23 09:34:47] financial_db> SELECT COUNT(DISTINCT company_id)
                                    FROM companies
[2026-04-23 09:34:47] 1 row retrieved starting from 1 in 390 ms (execution: 13 ms, fetching: 377 ms)
```

A continuació, validem la taula **credit_cards**:

```
SELECT COUNT(id)
FROM credit_cards;
```

Output | COUNT(id):int

1 row | CSV

	`COUNT(id)`
1	5000

Output | COUNT(id):int

```
[2026-04-23 00:48:20] financial_db> SELECT COUNT(id)
                                    FROM credit_cards
[2026-04-23 00:48:20] 1 row retrieved starting from 1 in 388 ms (execution: 13 ms, fetching: 375 ms)
```

```sql
SELECT COUNT(DISTINCT id)
FROM credit_cards;
```

Output | COUNT(DISTINCT id):int

1 row | CSV

	`COUNT(DISTINCT id)`
1	5000

Output | COUNT(DISTINCT id):int

```
[2026-04-23 09:37:52] financial_db> SELECT COUNT(DISTINCT id)
                                    FROM credit_cards
[2026-04-23 09:37:52] 1 row retrieved starting from 1 in 412 ms (execution: 30 ms, fetching: 382 ms)
```

I finalment, validem la taula **operations**:

```sql
SELECT COUNT(id)
FROM operations;
```

Output | COUNT(id):int

1 row | CSV

	`COUNT(id)`
1	100000

Output | COUNT(id):int

```
[2026-04-23 00:57:27] financial_db> SELECT COUNT(id)
                                    FROM operations
[2026-04-23 00:57:28] 1 row retrieved starting from 1 in 461 ms (execution: 128 ms, fetching: 333 ms)
```

```sql
SELECT COUNT(DISTINCT id)
FROM operations;
```

Output | COUNT(DISTINCT id):int

1 row | CSV

	`COUNT(DISTINCT id)`
1	100000

Output | COUNT(DISTINCT id):int

```
[2026-04-23 09:42:25] financial_db> SELECT COUNT(DISTINCT id)
                                    FROM operations
[2026-04-23 09:42:26] 1 row retrieved starting from 1 in 760 ms (execution: 419 ms, fetching: 341 ms)
```

Revisem els camps i modifiquem els noms dels que correspongui per normalitzar:

A la taula **users**:

```sql
ALTER TABLE users RENAME COLUMN id TO user_id;
```

Output | COUNT(DISTINCT id):int

```
[2026-04-23 10:00:17] financial_db> ALTER TABLE users RENAME COLUMN id TO user_id
[2026-04-23 10:00:17] completed in 54 ms
```

Validem els canvis:

```sql
SHOW COLUMNS FROM users;
```

Output | Result 18

11 rows | CSV

	Field	Type	Null	Key	Default	Extra
1	user_id	varchar(255)	YES		<null>	
2	name	varchar(255)	YES		<null>	
3	surname	varchar(255)	YES		<null>	
4	phone	varchar(255)	YES		<null>	
5	email	varchar(255)	YES		<null>	
6	birth_date	varchar(255)	YES		<null>	
7	country	varchar(255)	YES		<null>	
8	city	varchar(255)	YES		<null>	
9	postal_code	varchar(255)	YES		<null>	
10	address	varchar(255)	YES		<null>	
11	region	varchar(255)	YES		<null>	

Output | Result 18

```
[2026-04-23 10:06:29] financial_db> SHOW COLUMNS FROM users
[2026-04-23 10:06:30] 11 rows retrieved starting from 1 in 352 ms (execution: 8 ms, fetching: 344 ms)
```

Ara modifiquem la taula **credit_cards**:

```
ALTER TABLE credit_cards RENAME COLUMN id TO card_id;
```

Output | Result 15

```
[2026-04-23 10:04:13] financial_db> ALTER TABLE credit_cards RENAME COLUMN id TO card_id
[2026-04-23 10:04:13] completed in 32 ms
```

Validem els canvis:

```
SHOW COLUMNS FROM credit_cards;
```

Output | Result 19

9 rows | CSV

	Field	Type	Null	Key	Default	Extra
1	card_id	varchar(255)	YES		<null>	
2	user_id	varchar(255)	YES		<null>	
3	iban	varchar(255)	YES		<null>	
4	pan	varchar(255)	YES		<null>	
5	pin	varchar(255)	YES		<null>	
6	cvv	varchar(255)	YES		<null>	
7	track1	varchar(255)	YES		<null>	
8	track2	varchar(255)	YES		<null>	
9	expiring_date	varchar(255)	YES		<null>	

Output | Result 19

```
[2026-04-23 10:08:35] financial_db> SHOW COLUMNS FROM credit_cards
[2026-04-23 10:08:36] 9 rows retrieved starting from 1 in 360 ms (execution: 10 ms, fetching: 350 ms)
```

I finalment modifiquem la taula **operations**:

```
ALTER TABLE operations RENAME COLUMN business_id TO company_id;
```

Output | Result 15

```
[2026-04-23 10:05:13] financial_db> ALTER TABLE operations RENAME COLUMN business_id TO company_id
[2026-04-23 10:05:13] completed in 36 ms
```

```
ALTER TABLE operations RENAME COLUMN id TO operation_id;
```

Output | Result 20

```
[2026-04-23 10:14:00] financial_db> ALTER TABLE operations RENAME COLUMN id TO operation_id
[2026-04-23 10:14:00] completed in 34 ms
```

I validem els canvis:

```
SHOW COLUMNS FROM operations;
```

Output | Result 22

10 rows

	Field	Type	Null	Key	Default	Extra
1	operation_id	varchar(255)	YES		<null>	
2	card_id	varchar(255)	YES		<null>	
3	company_id	varchar(255)	YES		<null>	
4	timestamp	varchar(255)	YES		<null>	
5	amount	varchar(255)	YES		<null>	
6	declined	varchar(255)	YES		<null>	
7	product_ids	varchar(255)	YES		<null>	
8	user_id	varchar(255)	YES		<null>	
9	lat	varchar(255)	YES		<null>	
10	longitude	varchar(255)	YES		<null>	

Output | Result 22

```
[2026-04-23 10:15:44] financial_db> SHOW COLUMNS FROM operations
[2026-04-23 10:15:44] 10 rows retrieved starting from 1 in 364 ms (execution: 8 ms, fetching: 356 ms)
```

A la taula **companies** no ha fet falta modificar cap nom de cap camp.

A continuació, revisem i modifiquem els camps de dates de les diferents taules, per unificar els formats.

Modifiquem la taula **users**:

```
UPDATE Users
SET birth_date = DATE_FORMAT(STR_TO_DATE(birth_date, '%b %d, %Y'), '%Y-%m-%d')
WHERE birth_date LIKE '% % %';
```

Output | Result 22

```
[2026-04-23 11:05:56] financial_db> UPDATE Users
                                    SET birth_date = DATE_FORMAT(STR_TO_DATE(birth_date, '%b %d, %Y'), '%Y-%m-%d')
                                    WHERE birth_date LIKE '% % %'
[2026-04-23 11:05:57] 5,000 rows affected in 137 ms
```

Modifiquem la taula **credit_cards**:

```
UPDATE credit_cards
SET expiring_date = DATE_FORMAT(STR_TO_DATE(expiring_date, '%m/%d/%y'), '%Y-%m-%d')
WHERE expiring_date LIKE '%/%/%';
```

Output | Result 42

```
[2026-04-23 12:02:39] financial_db> UPDATE credit_cards
                                    SET expiring_date = DATE_FORMAT(STR_TO_DATE(expiring_date, '%m/%d/%y'), '%Y-%m-%d')
                                    WHERE expiring_date LIKE '%/%/%'
[2026-04-23 12:02:39] 5,000 rows affected in 157 ms
```

La taula **operations** no cal modificar-la ja que les dades tenen el format correcte, i la taula **companies** no té cap camp d'aquest estil.

Ara netejarem els valors de possibles espais en blanc.

Netegem la taula **users**:

```sql
UPDATE users SET
    user_id = TRIM(user_id),
    name = TRIM(name),
    surname = TRIM(surname),
    phone = TRIM(phone),
    email = LOWER(TRIM(email)),
    birth_date = TRIM(birth_date),
    country = TRIM(country),
    city = TRIM(city),
    postal_code = TRIM(postal_code),
    address = TRIM(address),
    region = TRIM(region);
```

Output | Result 33

```
[2026-04-23 11:44:58] financial_db> UPDATE users SET
                                    user_id = TRIM(user_id),
                                    name = TRIM(name),
                                    surname = TRIM(surname),
                                    phone = TRIM(phone),
                                    email = LOWER(TRIM(email)),
                                    birth_date = TRIM(birth_date),
                                    country = TRIM(country),
                                    city = TRIM(city),
                                    postal_code = TRIM(postal_code),
                                    address = TRIM(address),
                                    region = TRIM(region)
[2026-04-23 11:44:58] 5,000 rows affected in 23 ms
```

Netegem la taula **companies**:

```sql
UPDATE companies SET
    company_id = TRIM(company_id),
    company_name = TRIM(company_name),
    phone = TRIM(phone),
    email = LOWER(TRIM(email)),
    country = TRIM(country),
    website = LOWER(TRIM(website));
```

Output | Result 33

```
[2026-04-23 11:46:42] financial_db> UPDATE companies SET
                                    company_id = TRIM(company_id),
                                    company_name = TRIM(company_name),
                                    phone = TRIM(phone),
                                    email = LOWER(TRIM(email)),
                                    country = TRIM(country),
                                    website = LOWER(TRIM(website))
[2026-04-23 11:46:42] 100 rows affected in 5 ms
```

Netegem la taula **credit_cards**:

```sql
UPDATE credit_cards SET
    card_id = TRIM(card_id),
    user_id = TRIM(user_id),
    iban = TRIM(iban),
    pan = TRIM(pan),
    pin = TRIM(pin),
    cvv = TRIM(cvv),
    track1 = TRIM(track1),
    track2 = TRIM(track2),
    expiring_date = TRIM(expiring_date);
```

```
Output    Result 33
[2026-04-23 11:48:24] financial_db> UPDATE credit_cards SET
                                    card_id = TRIM(card_id),
                                    user_id = TRIM(user_id),
                                    iban = TRIM(iban),
                                    pan = TRIM(pan),
                                    pin = TRIM(pin),
                                    cvv = TRIM(cvv),
                                    track1 = TRIM(track1),
                                    track2 = TRIM(track2),
                                    expiring_date = TRIM(expiring_date)
[2026-04-23 11:48:24] 5,000 rows affected in 29 ms
```

Netegem la taula **operations**:

```
UPDATE operations SET
    operation_id = TRIM(operation_id),
    card_id = TRIM(card_id),
    company_id = TRIM(company_id),
    timestamp = TRIM(timestamp),
    amount = TRIM(amount),
    declined = TRIM(declined),
    product_ids = TRIM(product_ids),
    user_id = TRIM(user_id),
    lat = TRIM(lat),
    longitude = TRIM(longitude);
```

```
Output    Result 33
[2026-04-23 11:49:24] financial_db> UPDATE operations SET
                                    operation_id = TRIM(operation_id),
                                    card_id = TRIM(card_id),
                                    company_id = TRIM(company_id),
                                    timestamp = TRIM(timestamp),
                                    amount = TRIM(amount),
                                    declined = TRIM(declined),
                                    product_ids = TRIM(product_ids),
                                    user_id = TRIM(user_id),
                                    lat = TRIM(lat),
                                    longitude = TRIM(longitude)
[2026-04-23 11:49:25] 100,000 rows affected in 300 ms
```

Ara assignarem els valors que correspon a cada camp i les PK .

Comencem amb la taula **users**:

```
ALTER TABLE users
    MODIFY COLUMN user_id int NOT NULL AUTO_INCREMENT,
    MODIFY COLUMN name varchar(100) NOT NULL,
    MODIFY COLUMN surname varchar(100) NOT NULL,
    MODIFY COLUMN phone varchar(20),
    MODIFY COLUMN email varchar(150) NOT NULL,
    MODIFY COLUMN birth_date DATE NOT NULL,
    MODIFY COLUMN country varchar(100),
    MODIFY COLUMN city varchar(100),
    MODIFY COLUMN postal_code varchar(20),
    MODIFY COLUMN address varchar(255),
    MODIFY COLUMN region varchar(25),
    ADD PRIMARY KEY (user_id);
```

Output | Result 42

```
[2026-04-23 12:55:27] financial_db> ALTER TABLE users
                                    MODIFY COLUMN user_id int NOT NULL AUTO_INCREMENT,
                                    MODIFY COLUMN name varchar(100) NOT NULL,
                                    MODIFY COLUMN surname varchar(100) NOT NULL,
                                    MODIFY COLUMN phone varchar(20),
                                    MODIFY COLUMN email varchar(150) NOT NULL,
                                    MODIFY COLUMN birth_date DATE NOT NULL,
                                    MODIFY COLUMN country varchar(100),
                                    MODIFY COLUMN city varchar(100),
                                    MODIFY COLUMN postal_code varchar(20),
                                    MODIFY COLUMN address varchar(255),
                                    MODIFY COLUMN region varchar(25),
                                    ADD PRIMARY KEY (user_id)
[2026-04-23 12:55:27] 5,000 rows affected in 451 ms
```

Validem els canvis:

```
SHOW COLUMNS FROM users;
```

Output | Result 45

11 rows | CSV

	Field	Type	Null	Key	Default	Extra
1	user_id	int	NO	PRI	<null>	auto_increment
2	name	varchar(100)	NO		<null>	
3	surname	varchar(100)	NO		<null>	
4	phone	varchar(20)	YES		<null>	
5	email	varchar(150)	NO		<null>	
6	birth_date	date	NO		<null>	
7	country	varchar(100)	YES		<null>	
8	city	varchar(100)	YES		<null>	
9	postal_code	varchar(20)	YES		<null>	
10	address	varchar(255)	YES		<null>	
11	region	varchar(25)	YES		<null>	

Output | Result 45

```
[2026-04-23 12:57:02] financial_db> SHOW COLUMNS FROM users
[2026-04-23 12:57:03] 11 rows retrieved starting from 1 in 354 ms (execution: 9 ms, fetching: 345 ms)
```

Continuem amb la taula **companies**:

```
ALTER TABLE companies
    MODIFY COLUMN company_id varchar(10) NOT NULL,
    MODIFY COLUMN company_name varchar(100) NOT NULL,
    MODIFY COLUMN phone varchar(20),
    MODIFY COLUMN email varchar(150) NOT NULL,
    MODIFY COLUMN country varchar(100),
    MODIFY COLUMN website varchar(250),
    ADD PRIMARY KEY (company_id);
```

Output | Result 45

```
[2026-04-23 12:59:19] financial_db> ALTER TABLE companies
                                    MODIFY COLUMN company_id varchar(10) NOT NULL,
                                    MODIFY COLUMN company_name varchar(100) NOT NULL,
                                    MODIFY COLUMN phone varchar(20),
                                    MODIFY COLUMN email varchar(150) NOT NULL,
                                    MODIFY COLUMN country varchar(100),
                                    MODIFY COLUMN website varchar(250),
                                    ADD PRIMARY KEY (company_id)
[2026-04-23 12:59:19] 100 rows affected in 72 ms
```

Validem els canvis:

```sql
SHOW COLUMNS FROM companies;
```

Output | Result 47

6 rows

	Field	Type	Null	Key	Default	Extra
1	company_id	varchar(10)	NO	PRI	<null>	
2	company_name	varchar(100)	NO		<null>	
3	phone	varchar(20)	YES		<null>	
4	email	varchar(150)	NO		<null>	
5	country	varchar(100)	YES		<null>	
6	website	varchar(250)	YES		<null>	

Output | Result 47

```
[2026-04-23 13:00:20] financial_db> SHOW COLUMNS FROM companies
[2026-04-23 13:00:20] 6 rows retrieved starting from 1 in 355 ms (execution: 9 ms, fetching: 346 ms)
```

Ara amb la taula **credit_cards**:

```sql
ALTER TABLE credit_cards
    MODIFY COLUMN card_id varchar(10) NOT NULL,
    MODIFY COLUMN user_id int NOT NULL,
    MODIFY COLUMN iban varchar(34) NOT NULL,
    MODIFY COLUMN pan varchar(20) NOT NULL,
    MODIFY COLUMN pin varchar(5) NOT NULL,
    MODIFY COLUMN cvv varchar(4) NOT NULL,
    MODIFY COLUMN track1 varchar(250),
    MODIFY COLUMN track2 varchar(250),
    MODIFY COLUMN expiring_date DATE NOT NULL,
    ADD PRIMARY KEY (card_id);
```

Output | Result 47

```
[2026-04-23 13:01:36] financial_db> ALTER TABLE credit_cards
                                    MODIFY COLUMN card_id varchar(10) NOT NULL,
                                    MODIFY COLUMN user_id int NOT NULL,
                                    MODIFY COLUMN iban varchar(34) NOT NULL,
                                    MODIFY COLUMN pan varchar(20) NOT NULL,
                                    MODIFY COLUMN pin varchar(5) NOT NULL,
                                    MODIFY COLUMN cvv varchar(4) NOT NULL,
                                    MODIFY COLUMN track1 varchar(250),
                                    MODIFY COLUMN track2 varchar(250),
                                    MODIFY COLUMN expiring_date DATE NOT NULL,
                                    ADD PRIMARY KEY (card_id)
[2026-04-23 13:01:37] 5,000 rows affected in 210 ms
```

Validem els canvis:

```sql
SHOW COLUMNS FROM credit_cards;
```

Output | Result 49

9 rows

	Field	Type	Null	Key	Default	Extra
1	card_id	varchar(10)	NO	PRI	<null>	
2	user_id	int	NO		<null>	
3	iban	varchar(34)	NO		<null>	
4	pan	varchar(20)	NO		<null>	
5	pin	varchar(5)	NO		<null>	
6	cvv	varchar(4)	NO		<null>	
7	track1	varchar(250)	YES		<null>	
8	track2	varchar(250)	YES		<null>	
9	expiring_date	date	NO		<null>	

```
Output    Result 49
[2026-04-23 13:02:39] financial_db> SHOW COLUMNS FROM credit_cards
[2026-04-23 13:02:40] 9 rows retrieved starting from 1 in 371 ms (execution: 19 ms, fetching: 352 ms)
```

I finalment amb la taula **operations**:

```sql
ALTER TABLE operations
    MODIFY COLUMN operation_id varchar(40) NOT NULL,
    MODIFY COLUMN card_id varchar(10) NOT NULL,
    MODIFY COLUMN company_id varchar(10) NOT NULL,
    MODIFY COLUMN timestamp datetime NOT NULL,
    MODIFY COLUMN amount decimal(10,2) NOT NULL,
    MODIFY COLUMN declined boolean NOT NULL,
    MODIFY COLUMN product_ids varchar(250),
    MODIFY COLUMN user_id int NOT NULL,
    MODIFY COLUMN lat decimal(11,8),
    MODIFY COLUMN longitude decimal(11,8),
    ADD PRIMARY KEY (operation_id);
```

Degut un error de càlcul en els camps lat i longitude, hem de tornar a llençar el codi (excepte l'assignació del primary key ja que ja s'ha assignat):

```sql
ALTER TABLE operations
    MODIFY COLUMN operation_id varchar(40) NOT NULL,
    MODIFY COLUMN card_id varchar(10) NOT NULL,
    MODIFY COLUMN company_id varchar(10) NOT NULL,
    MODIFY COLUMN timestamp datetime NOT NULL,
    MODIFY COLUMN amount decimal(10,2) NOT NULL,
    MODIFY COLUMN declined boolean NOT NULL,
    MODIFY COLUMN product_ids varchar(250),
    MODIFY COLUMN user_id int NOT NULL,
    MODIFY COLUMN lat decimal(18,15),
    MODIFY COLUMN longitude decimal(18,15);
```

```
Output    Result 49
[2026-04-23 13:13:54] financial_db> ALTER TABLE operations
                                    MODIFY COLUMN operation_id varchar(40) NOT NULL,
                                    MODIFY COLUMN card_id varchar(10) NOT NULL,
                                    MODIFY COLUMN company_id varchar(10) NOT NULL,
                                    MODIFY COLUMN timestamp datetime NOT NULL,
                                    MODIFY COLUMN amount decimal(10,2) NOT NULL,
                                    MODIFY COLUMN declined boolean NOT NULL,
                                    MODIFY COLUMN product_ids varchar(250),
                                    MODIFY COLUMN user_id int NOT NULL,
                                    MODIFY COLUMN lat decimal(18,15),
                                    MODIFY COLUMN longitude decimal(18,15)
[2026-04-23 13:13:54] completed in 19 ms
```

I validem els canvis:

```sql
SHOW COLUMNS FROM operations;
```

Output Result 54

10 rows CSV

	Field	Type	Null	Key	Default	Extra
1	operation_id	varchar(40)	NO	PRI	<null>	
2	card_id	varchar(10)	NO		<null>	
3	company_id	varchar(10)	NO		<null>	
4	timestamp	datetime	NO		<null>	
5	amount	decimal(10,2)	NO		<null>	
6	declined	tinyint(1)	NO		<null>	
7	product_ids	varchar(250)	YES		<null>	
8	user_id	int	NO		<null>	
9	lat	decimal(18,15)	YES		<null>	
10	longitude	decimal(18,15)	YES		<null>	

```
Output    Result 54
[2026-04-23 13:15:02] financial_db> SHOW COLUMNS FROM operations
[2026-04-23 13:15:03] 10 rows retrieved starting from 1 in 363 ms (execution: 14 ms, fetching: 349 ms)
```

Un cop fet això, assignarem les FK a les taules.

Comencem amb la FK per la taula **users**:

```
ALTER TABLE operations
    ADD CONSTRAINT fk_operations_users
    FOREIGN KEY (user_id) REFERENCES users(user_id);
```

```
Output    Result 54
[2026-04-23 13:23:41] financial_db> ALTER TABLE operations
                                    ADD CONSTRAINT fk_operations_users
                                    FOREIGN KEY (user_id) REFERENCES users(user_id)
[2026-04-23 13:23:43] 100,000 rows affected in 1 s 688 ms
```

Continuem amb la FK per la taula **companies**:

```
ALTER TABLE operations
    ADD CONSTRAINT fk_operations_companies
    FOREIGN KEY (company_id) REFERENCES companies(company_id);
```

```
Output    Result 54
[2026-04-23 13:24:59] financial_db> ALTER TABLE operations
                                    ADD CONSTRAINT fk_operations_companies
                                    FOREIGN KEY (company_id) REFERENCES companies(company_id)
[2026-04-23 13:25:01] 100,000 rows affected in 2 s 56 ms
```

Ara amb la FK per la taula **credit_cards**:

```
ALTER TABLE operations
    ADD CONSTRAINT fk_operations_cards
    FOREIGN KEY (card_id) REFERENCES credit_cards(card_id);
```

```
Output    Result 54
[2026-04-23 13:25:48] financial_db> ALTER TABLE operations
                                    ADD CONSTRAINT fk_operations_cards
                                    FOREIGN KEY (card_id) REFERENCES credit_cards(card_id)
[2026-04-23 13:25:50] 100,000 rows affected in 2 s 429 ms
```

I validem els canvis:

```
SHOW COLUMNS FROM operations;
```

Output Result 59

10 rows CSV

	Field	Type	Null	Key	Default	Extra
1	operation_id	varchar(40)	NO	PRI	<null>	
2	card_id	varchar(10)	NO	MUL	<null>	
3	company_id	varchar(10)	NO	MUL	<null>	
4	timestamp	datetime	NO		<null>	
5	amount	decimal(10,2)	NO		<null>	
6	declined	tinyint(1)	NO		<null>	
7	product_ids	varchar(250)	YES		<null>	
8	user_id	int	NO	MUL	<null>	
9	lat	decimal(18,15)	YES		<null>	
10	longitude	decimal(18,15)	YES		<null>	

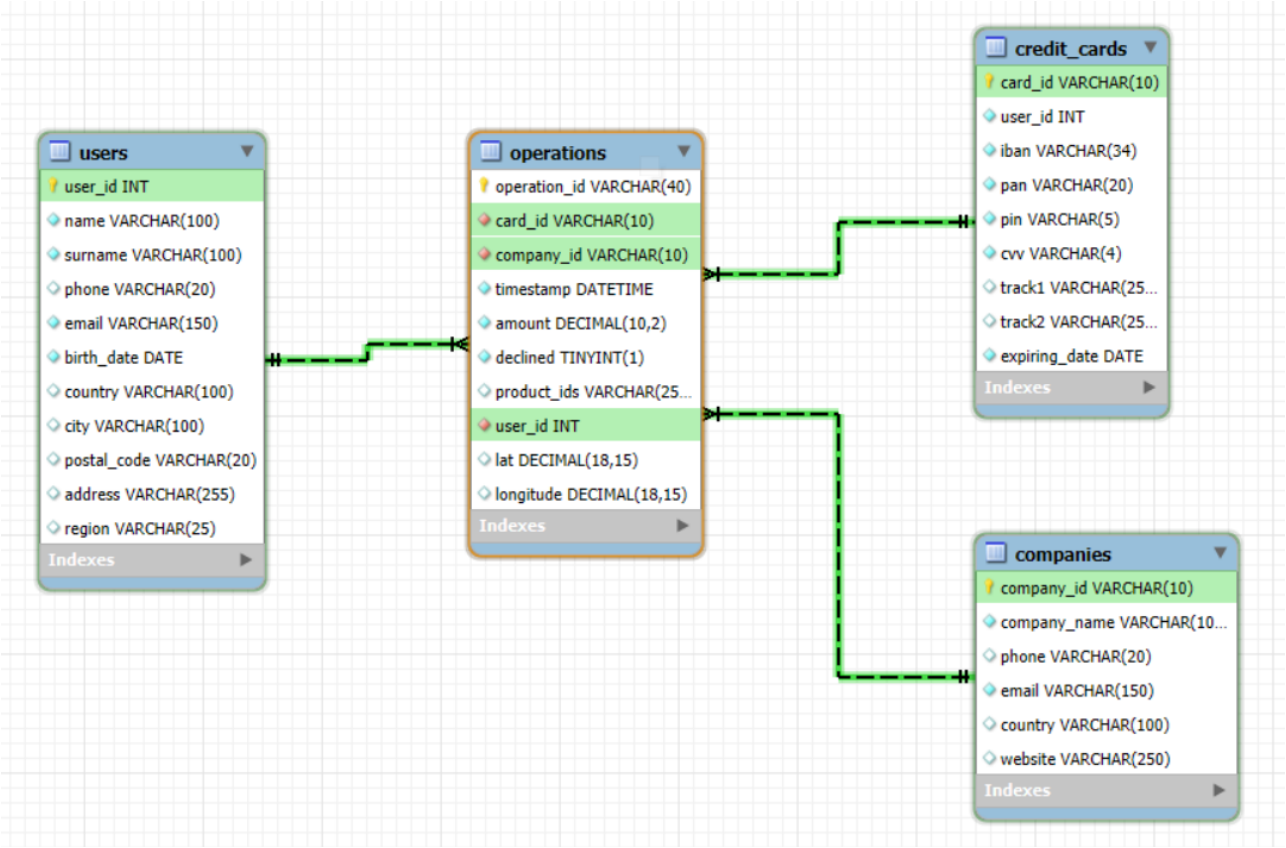
```
Output    Result 59
[2026-04-23 13:27:00] financial_db> SHOW COLUMNS FROM operations
[2026-04-23 13:27:00] 10 rows retrieved starting from 1 in 363 ms (execution: 10 ms, fetching: 353 ms)
```

Amb aquestes accions, hem consolidat l'esquema de la base de dades seguint un model en estrella. En aquesta arquitectura, la taula de fets (**operations**) actua com el nucli central que registra les transaccions, mentre que les taules de dimensions (**users**, **credit_cards** i **companies**) aporten el context descriptiu.

Aquestes dimensions estan vinculades a la taula de fets mitjançant FK (user_id, card_id i company_id), garantint una estructura optimitzada per a la consulta i l'anàlisi de dades financeres.

Les relacions son de molts a un (N:1) entre **operations** i **users**, **operations** i **credit_cards** i **operations** i **companies**.

El diagrama corresponent seria aquest:



Exercici 9
Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

Seleccionem el id d'usuari per evitar possibles confusions per duplicitat, el nom i cognom i el número de transaccions per usuari (dada que traiem de la subconsulta).
En la subconsulta hem seleccionat el id de l'usuari i hem comptat el número d'operacions, agrupant per l'id i filtrant amb HAVING.

```
SELECT u.user_id AS ID_Usuari, u.name AS Nom, u.surname AS Cognom, t.Numero_Transaccions
FROM users AS u
JOIN (SELECT o.user_id, COUNT(o.operation_id) AS Numero_Transaccions
        FROM operations AS o
        WHERE o.declined = 0
        GROUP BY o.user_id
        HAVING COUNT(o.operation_id) > 80) AS t
ON u.user_id = t.user_id;
```

Output | Realitza una subcons...ant almenys 2 taules.

3 rows

	ID_Usuari	Nom	Cognom	Numero_Transaccions
1	185	Molly	Gilliam	107
2	289	Dxwgi	Hwcru	91
3	318	Bnyr	Astuw	86

Output | Realitza una subcons...ant almenys 2 taules.

```
[2026-04-24 17:42:36] financial_db> SELECT u.user_id AS ID_Usuari, u.name AS Nom, u.surname AS Cognom, t.Numero_Transaccions
                                    FROM users AS u
                                    JOIN (SELECT o.user_id, COUNT(o.operation_id) AS Numero_Transaccions
                                            FROM operations AS o
                                            WHERE o.declined = 0
                                            GROUP BY o.user_id
                                            HAVING COUNT(o.operation_id) > 80) AS t
                                    ON u.user_id = t.user_id
[2026-04-24 17:42:37] 3 rows retrieved starting from 1 in 959 ms (execution: 625 ms, fetching: 334 ms)
```

Exercici 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

Seleccionem el nom de l'empresa, el id de la targeta i el IBAN. Calculem la mitjana amb AVG i el ROUND amb dos decimals. Agrupem per id de la targeta, el nom i el IBAN.

```
SELECT c.company_name AS Nom_Empresa, cc.card_id AS ID_Targeta, cc.iban AS IBAN, ROUND(AVG(o.amount),2) AS Mitjana_Quantitat
FROM credit_cards AS cc
JOIN operations AS o ON cc.card_id = o.card_id
JOIN companies AS c ON c.company_id = o.company_id
WHERE c.company_name = 'Donec Ltd' AND o.declined = 0
GROUP BY cc.card_id,c.company_name, cc.iban;
```

Output | Mostra la mitjana d'...tza almenys 2 taules.

370 rows

CSV

	Nom_Empresa	ID_Targeta	IBAN	Mitjana_Quantitat
1	Donec Ltd	CcS-5928	XX9114064011255863075868 05	356.25
2	Donec Ltd	CcU-4009	SK9446370242474562577506	142.96
3	Donec Ltd	CcS-6066	XX7767529178459529755556 40	257.37
4	Donec Ltd	CcS-8452	XX4138273622897193049089 90	139.59
5	Donec Ltd	CcS-7810	XX3477872460707696107803 08	240.41
6	Donec Ltd	CcS-6547	XX6887684365430908948546 02	188.58
7	Donec Ltd	CcU-4135	MK28368851538688349	439.39
8	Donec Ltd	CcU-4541	PL7624928356685267634340 4576	541.56

Output | Mostra la mitjana d'...tza almenys 2 taules.

```
[2026-04-27 13:46:15] financial_db> SELECT c.company_name AS Nom_Empresa, cc.card_id AS ID_Targeta, cc.iban AS IBAN, ROUND(AVG(o
.amount),2) AS Mitjana_Quantitat
                                    FROM credit_cards AS cc
                                    JOIN operations AS o ON cc.card_id = o.card_id
                                    JOIN companies AS c ON c.company_id = o.company_id
                                    WHERE c.company_name = 'Donec Ltd' AND o.declined = 0
                                    GROUP BY cc.card_id,c.company_name, cc.iban
[2026-04-27 13:46:16] 370 rows retrieved starting from 1 in 432 ms (execution: 37 ms, fetching: 395 ms)
```

Nivell 2

Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

Es seleccionen els dies utilitzant DATE i es calcula la suma dels ingressos amb un SUM i ROUND amb dos decimals.
S'agrupa per data, s'ordena pel total d'ingressos i es limiten els resultats amb LIMIT.

```
SELECT DATE(o.timestamp) AS Dies, ROUND(SUM(o.amount),2) AS Total_Vendes
FROM operations AS o
WHERE o.declined = 0
GROUP BY DATE (o.timestamp)
ORDER BY Total_Vendes DESC
LIMIT 5;
```

Output | Mostra la data de ca... total de les vendes.

5 rows

	Dies	Total_Vendes
1	2022-12-13	14337.44
2	2019-11-18	13591.32
3	2023-02-20	13332.59
4	2017-12-20	13318.43
5	2019-03-18	12680.95

Output | Mostra la data de ca... total de les vendes.

```
[2026-04-24 17:53:35] financial_db> SELECT DATE(o.timestamp) AS Dies, ROUND(SUM(o.amount),2) AS Total_Vendes
                                    FROM operations AS o
                                    WHERE o.declined = 0
                                    GROUP BY DATE (o.timestamp)
                                    ORDER BY Total_Vendes DESC
                                    LIMIT 5
[2026-04-24 17:53:36] 5 rows retrieved starting from 1 in 462 ms (execution: 90 ms, fetching: 372 ms)
```

Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

Es seleccionen els camps que ens demanen, utilitzem DATE per les dates, utilitzem ROUND amb dos decimals per la quantitat. Filtrem pels valors que ens indiquen i per les dates especificades. Ordenem de major a menor a partir de la quantitat.

```
SELECT c.company_name AS Companyia, c.phone AS Telèfon, c.country AS País, DATE(o.timestamp) AS Data, ROUND(o.amount,2) AS Quantitat
FROM companies AS c
JOIN operations AS o ON c.company_id = o.company_id
WHERE (ROUND(o.amount,2) BETWEEN 350 AND 400) AND (DATE(o.timestamp) = '2015-04-29' OR DATE(o.timestamp) = '2018-07-20' OR DATE(o.timestamp) = '2024-03-13') AND
 o.declined = 0
ORDER BY ROUND(o.amount,2) DESC;
```

Output | Ordena els resultats...or a menor quantitat.

8 rows

	Companyia	Telèfon	País	Data	Quantitat
1	Aliquam PC	01 45 73 52 16	Germany	2024-03-13	399.84
2	Auctor Mauris Vel LLP	08 09 28 74 14	United States	2018-07-20	399.51
3	At Pede Corp.	06 14 48 33 15	Italy	2015-04-29	390.69
4	Aliquam PC	01 45 73 52 16	Germany	2024-03-13	388.29
5	Orci Adipiscing Limited	03 18 00 77 81	United Kingdom	2018-07-20	373.71
6	Fringilla LLC	08 29 15 93 57	New Zealand	2015-04-29	367.62
7	Pede Cum Ltd	07 62 26 48 38	Norway	2018-07-20	356.87
8	Auctor Mauris Vel LLP	08 09 28 74 14	United States	2024-03-13	353.75

```
Output   Ordena els resultats...or a menor quantitat.
[2026-04-24 17:56:33] financial_db> SELECT c.company_name AS Companyia, c.phone AS Telèfon, c.country AS País, DATE(o.timestamp) AS Data, ROUND(o.amount,2) AS Quantitat
                                    FROM companies AS c
                                    JOIN operations AS o ON c.company_id = o.company_id
                                    WHERE (ROUND(o.amount,2) BETWEEN 350 AND 400) AND (DATE(o.timestamp) = '2015-04-29' OR DATE(o.timestamp) = '2018-07-20' OR
                                     DATE(o.timestamp) = '2024-03-13') AND o.declined = 0
                                    ORDER BY ROUND(o.amount,2) DESC
[2026-04-24 17:56:33] 8 rows retrieved starting from 1 in 425 ms (execution: 66 ms, fetching: 359 ms)
```

Exercici 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen més de 400 transaccions o menys.

Seleccionem per id de l'empresa i el seu nom (per evitar posibles confusions amb noms duplicats), contem el nombre de transaccions realitzats per empresa i generem una columna extra on s'indica si l'empresa ha realitzat 400 transaccions, o menys de 400 transaccions. Com que no especifica on s'ha d'agrupar les empreses que tinguin just 400 transaccions, es crea una categoria.

```
SELECT c.company_id AS ID_Empresa, c.company_name AS Nom, COUNT(o.operation_id) AS Operacions,
       CASE
            WHEN COUNT(o.operation_id) < 400 THEN 'Menys de 400 transaccions'
            WHEN COUNT(o.operation_id) = 400 THEN '400 transaccions'
            ELSE 'Més de 400 transaccions'
       END AS Transaccions
FROM companies AS c
JOIN operations AS o ON c.company_id = o.company_id
WHERE o.declined = 0
GROUP BY c.company_id, c.company_name;
```

Output 400 transaccions o menys.

100 rows

	ID_Empresa	Nom	Operacions	Transaccions
1	b-2458	Eget Tincidunt Dui Institute	1522	Més de 400 transaccions
2	b-2370	Non Justo Corp.	1476	Més de 400 transaccions
3	b-2390	Neque Tellus Imperdiet Corp.	422	Més de 400 transaccions
4	b-2230	Fusce Corp.	445	Més de 400 transaccions
5	b-2266	Mus Aenean Eget Foundation	1563	Més de 400 transaccions
6	b-2598	Aliquam Iaculis Lacus Corp.	422	Més de 400 transaccions
7	b-2546	Lorem Ipsum Dolor Corp.	1465	Més de 400 transaccions
8	b-2526	Amet Luctus Vulputate Foundation	410	Més de 400 transaccions

```
Output   400 transaccions o menys.
[2026-04-24 17:58:36] financial_db> SELECT c.company_id AS ID_Empresa, c.company_name AS Nom, COUNT(o.operation_id) AS Operacions,
                                    CASE
                                         WHEN COUNT(o.operation_id) < 400 THEN 'Menys de 400 transaccions'
                                         WHEN COUNT(o.operation_id) = 400 THEN '400 transaccions'
                                         ELSE 'Més de 400 transaccions'
                                    END AS Transaccions
                                FROM companies AS c
                                JOIN operations AS o ON c.company_id = o.company_id
                                WHERE o.declined = 0
                                GROUP BY c.company_id, c.company_name
[2026-04-24 17:58:36] 100 rows retrieved starting from 1 in 629 ms (execution: 290 ms, fetching: 339 ms)
```

Exercici 4

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

S'elimina el registre indicat:

```
DELETE FROM operations WHERE operation_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
Output   Elimina de la taula ... de la base de dades.
[2026-04-24 18:03:48] financial_db> DELETE FROM operations WHERE operation_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'
[2026-04-24 18:03:48] 1 row affected in 7 ms
```

I es valida que s'hagi eliminat correctament:

```
SELECT * FROM operations WHERE operation_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Output | financial_db.operations

0 rows | Tx: Auto | DDL | CSV

operation_id	card_id	company_id	timestamp	amount

Output | financial_db.operations

```
[2026-04-27 11:47:01] financial_db> SELECT * FROM operations WHERE operation_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'
[2026-04-27 11:47:01] 0 rows retrieved in 419 ms (execution: 63 ms, fetching: 356 ms)
```

Exercici 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

Es crea una la vista VistaMarketing amb els camps sol·licitats. Es calcula la mitjana amb AVG i ROUND amb dos decimals.

```
CREATE VIEW VistaMarketing AS
SELECT c.company_id AS ID_Empresa, c.company_name AS Nom, c.phone AS Telèfon, c.country AS País, ROUND(AVG(o.amount),2) AS Mitjana_de_Compra
FROM companies AS c
JOIN operations AS o ON c.company_id = o.company_id
WHERE o.declined = 0
GROUP BY c.company_id, c.company_name, c.phone, c.country;
```

```
[2026-04-24 18:07:15] financial_db> CREATE VIEW VistaMarketing AS
                                    SELECT c.company_id AS ID_Empresa, c.company_name AS Nom, c.phone AS Telèfon, c.country AS País, ROUND(AVG(o.amount),2) AS
                                     Mitjana_de_Compra
                                    FROM companies AS c
                                    JOIN operations AS o ON c.company_id = o.company_id
                                    WHERE o.declined = 0
                                    GROUP BY c.company_id, c.company_name, c.phone, c.country
[2026-04-24 18:07:15] completed in 21 ms
```

Per veure la vista, s'executa el següent codi:

```
SELECT * FROM VistaMarketing
ORDER BY Mitjana_de_Compra DESC;
```

Output | financial_db.vistamarketing

100 rows | Tx: Auto | DDL

	ID_Empresa	Nom	Telèfon	País	Mitjana_de_Compra
1	b-2222	Ac Fermentum Incorporated	06 85 56 52 33	Germany	284.91
2	b-2282	Pretium Neque Corp.	07 77 48 55 28	Australia	275.58
3	b-2422	Urna Convallis Associates	06 01 24 77 04	United States	273.57
4	b-2538	At Associates	09 56 61 10 65	New Zealand	272.74
5	b-2498	Metus Vitae Associates	08 25 44 40 66	Australia	270.05

Output | financial_db.vistamarketing

```
[2026-04-24 18:09:53] financial_db> SELECT * FROM VistaMarketing
                                    ORDER BY Mitjana_de_Compra DESC
[2026-04-24 18:09:54] 100 rows retrieved starting from 1 in 855 ms (execution: 483 ms, fetching: 372 ms)
```

Nivell 3

Exercici 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

Quantes targetes estan actives?

Amb WITH definim dues CTE's per generar la taula. La primera CTE classifica amb ROW_NUMBER per l'id de la tarjeta i l'ordre cronològic de transaccions. La segona CTE calcula si les tarjeta son actives o inactives calculant que hi hagi tres operacions i que les tres operacions siguin declinades.

```sql
WITH OrdreOperacions AS (SELECT card_id, declined,
                                ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) AS posicio
                                FROM operations),
EstatTargetes AS (SELECT card_id,
                    CASE
                        WHEN SUM(declined) = 3 AND COUNT(*) = 3 THEN 'Inactiva'
                        ELSE 'Activa'
                    END AS estat
                    FROM OrdreOperacions
                    WHERE posicio <= 3
                    GROUP BY card_id)

-- 👉 Quantes targetes estan actives?

SELECT COUNT(*) AS Total_Targetes_Actives
FROM EstatTargetes
WHERE estat = 'Activa';
```

Output | estat declinades ale...aquesta taula respon:

1 row

	Total_Targetes_Actives
1	4995

Output | estat declinades ale...aquesta taula respon:

```
[2026-04-24 18:11:56] financial_db> WITH OrdreOperacions AS (SELECT card_id, declined,
                                                                ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) AS posicio
                                                                FROM operations),
                                    EstatTargetes AS (SELECT card_id,
                                                        CASE
                                                            WHEN SUM(declined) = 3 AND COUNT(*) = 3 THEN 'Inactiva'
                                                            ELSE 'Activa'
                                                        END AS estat
                                                        FROM OrdreOperacions
                                                        WHERE posicio <= 3
                                                        GROUP BY card_id)

                                    -- 👉 Quantes targetes estan actives?

                                    SELECT COUNT(*) AS Total_Targetes_Actives
                                    FROM EstatTargetes
                                    WHERE estat = 'Activa'
[2026-04-24 18:11:56] 1 row retrieved starting from 1 in 535 ms (execution: 213 ms, fetching: 322 ms)
```

Exercici 2

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids.

Primer de tot, revisem amb l'editor el tipus d'arxiu .csv. Té la codificació UTF-8, i el final de línia Unix (LF).

Unix (LF) | UTF-8

També fem un simulacre de càrrega de dades per previsualitzar:

Data Preview

file: N3.Ex.2__products.csv format: CSV

	id	product_name	price	colour	weight	warehouse_id
1	1	Direwolf Stannis	$161.11	#7c7c7c	1	WH-4
2	2	Tarly Stark	$9.24	#919191	2	WH-3
3	3	duel tourney Lannister	$171.13	#d8d8d8	1.5	WH-2
4	4	warden south duel	$71.89	#111111	3	WH-1

1. Estratègia d'importació i normalització inicial
Per garantir la integritat de la importació des de l'arxiu .csv, primer crearem la taula **products** utilitzant el tipus de dada VARCHAR(255) per a totes les columnes i sense assignar cap Clau Primària (PK). Aquesta fase de "taula de transició" evitarà errors de format durant la càrrega inicial. Un cop creada, procedirem a la importació de dades mitjançant l'ordre LOAD DATA.

2. Validació i consistència de nomenclatura
Un cop les dades estiguin carregades, validarem que el nombre de registres coincideixi amb l'origen. Posteriorment, anomenarem el camp id a product_id per mantenir una nomenclatura coherent amb la resta de la base de dades i facilitar la normalització.

3. Depuració i neteja de dades (Data Cleaning)
Abans de qualsevol canvi estructural, aplicarem un procés de neteja:

Eliminarem espais en blanc innecessaris amb TRIM.
Netejarem el camp price eliminant el símbol de divisa ($).
Corregirem errors de format al camp warehouse_id, normalitzant els dobles guions (--) a un de sol.

4. Definició d'estructures i PK
Amb les dades netes, modificarem els tipus de dades (MODIFY COLUMN) als seus formats correctes: INT per a l'identificador, i DECIMAL per al pes i el preu, assegurant que no es perdi informació decimal. Finalment, assignarem product_id com a PK.

5. Resolució de la relació molts a molts
En analitzar les dades de l'arxiu i la taula de fets **operations**, s'ha detectat que el camp product_ids conté llistats de múltiples productes separats per comes. Això impedeix la creació d'una Clau Forana (FK) directa. Per tant, crearem una taula intermèdia anomenada **operations_products**. Mitjançant una operació de split de dades (usant SUBSTRING_INDEX) i LENGHT per descobrir quants números hi ha com a màxim al camp products_id, traspassarem cada ID de producte a una fila independent, relacionant-lo amb el seu operation_id.

6. Establiment de relacions (Foreign Keys)
Finalment, un cop les dades estan normalitzades en la nova taula intermèdia, assignarem les FK corresponents que connectaran **operations_products** amb **products** i amb **operations**, garantint així la integritat referencial de tot el model.

Comencem creant la taula:

```
USE financial_db;
```

```
[2026-04-24 14:31:03] financial_db> USE financial_db
[2026-04-24 14:31:03] completed in 6 ms
```

```
CREATE TABLE IF NOT EXISTS products
(
    id varchar(255),
    product_name varchar(255),
    price varchar(255),
    colour varchar(255),
    weight varchar(255),
    warehouse_id varchar(255)
);
```

```
[2026-04-24 14:32:19] financial_db> CREATE TABLE IF NOT EXISTS products
                                    (
                                        id varchar(255),
                                        product_name varchar(255),
                                        price varchar(255),
                                        colour varchar(255),
                                        weight varchar(255),
                                        warehouse_id varchar(255)
                                    )
[2026-04-24 14:32:19] completed in 290 ms
```

Ara carreguem les dades de l'arxiu products:

```
LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N3.Ex.2__products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

```
[2026-04-24 14:32:19] completed in 290 ms
[2026-04-24 14:34:34] financial_db> LOAD DATA LOCAL INFILE 'C:/Users/USUARIO/Documents/ITADataAnalysis/Data/N3.Ex.2__products.csv'
                                    INTO TABLE products
                                    FIELDS TERMINATED BY ','
                                    LINES TERMINATED BY '\n'
                                    IGNORE 1 ROWS
[2026-04-24 14:34:35] 100 rows affected in 34 ms
```

Validem la càrrega de dades:

```
SELECT COUNT(id)
FROM products;
```

Output | COUNT(id):int

1 row

	`COUNT(id)`
1	100

Output | COUNT(id):int

```
[2026-04-24 14:36:24] financial_db> SELECT COUNT(id)
                                    FROM products
[2026-04-24 14:36:24] 1 row retrieved starting from 1 in 432 ms (execution: 18 ms, fetching: 414 ms)
```

```
SELECT COUNT(DISTINCT id)
FROM products;
```

Output | COUNT(DISTINCT id):int

1 row

	`COUNT(DISTINCT id)`
1	100

Output | COUNT(DISTINCT id):int

```
[2026-04-24 14:37:46] financial_db> SELECT COUNT(DISTINCT id)
                                    FROM products
[2026-04-24 14:37:46] 1 row retrieved starting from 1 in 387 ms (execution: 17 ms, fetching: 370 ms)
```

Procedim a netejar les dades. Primer eliminem el símbol $:

```
UPDATE products SET price = REPLACE(price, '$', '')
WHERE LOCATE('$', price) > 0;
```

```
[2026-04-24 14:51:13] financial_db> UPDATE products SET price = REPLACE(price, '$', '')
                                    WHERE LOCATE('$', price) > 0
[2026-04-24 14:51:13] 100 rows affected in 22 ms
```

I validem el canvi realitzat:

```sql
SELECT price
FROM products;
```

Output | financial_db.products

100 rows | Tx: Auto | DDL

	price
1	161.11
2	9.24
3	171.13
4	71.89
5	171.22
6	136.60
7	63.33
8	32.37

Output | financial_db.products

```
[2026-04-24 14:53:26] financial_db> SELECT price
                                    FROM products
[2026-04-24 14:53:27] 100 rows retrieved starting from 1 in 390 ms (execution: 7 ms, fetching: 383 ms)
```

I ara procedirem a substituir el guio doble (–) per un sòl guió (-):

```sql
UPDATE products SET warehouse_id = REPLACE(warehouse_id, '--', '-')
WHERE LOCATE('--', warehouse_id) > 0;
```

```
[2026-04-24 14:56:06] financial_db> UPDATE products SET warehouse_id = REPLACE(warehouse_id, '--', '-')
                                    WHERE LOCATE('--', warehouse_id) > 0
[2026-04-24 14:56:06] 95 rows affected in 23 ms
```

Validem el canvi:

```sql
SELECT warehouse_id
FROM products;
```

Output | financial_db.products

100 rows | Tx: Auto | DDL

	warehouse_id
1	WH-4
2	WH-3
3	WH-2
4	WH-1
5	WH-0
6	WH-1
7	WH-2
8	WH-3

Output | financial_db.products

```
[2026-04-24 14:57:03] financial_db> SELECT warehouse_id
                                    FROM products
[2026-04-24 14:57:04] 100 rows retrieved starting from 1 in 404 ms (execution: 10 ms, fetching: 394 ms)
```

Ara netejarem les dades d'espais en blanc:

```sql
UPDATE products SET
    id = TRIM(id),
    product_name = TRIM(product_name),
    price = TRIM(price),
    colour = TRIM(colour),
    weight = TRIM(weight),
    warehouse_id = TRIM(warehouse_id)
WHERE
    LENGTH(id) <> LENGTH(TRIM(id)) OR
    LENGTH(product_name) <> LENGTH(TRIM(product_name)) OR
    LENGTH(price) <> LENGTH(TRIM(price)) OR
    LENGTH(colour) <> LENGTH(TRIM(colour)) OR
    LENGTH(weight) <> LENGTH(TRIM(weight)) OR
    LENGTH(warehouse_id) <> LENGTH(TRIM(warehouse_id));
```

```
[2026-04-24 15:20:22] financial_db> UPDATE products SET
                                        id = TRIM(id),
                                        product_name = TRIM(product_name),
                                        price = TRIM(price),
                                        colour = TRIM(colour),
                                        weight = TRIM(weight),
                                        warehouse_id = TRIM(warehouse_id)
                                    WHERE
                                        LENGTH(id) <> LENGTH(TRIM(id)) OR
                                        LENGTH(product_name) <> LENGTH(TRIM(product_name)) OR
                                        LENGTH(price) <> LENGTH(TRIM(price)) OR
                                        LENGTH(colour) <> LENGTH(TRIM(colour)) OR
                                        LENGTH(weight) <> LENGTH(TRIM(weight)) OR
                                        LENGTH(warehouse_id) <> LENGTH(TRIM(warehouse_id))
[2026-04-24 15:20:22] completed in 5 ms
```

Ara, canviarem el nom del camp id:

```sql
ALTER TABLE products RENAME COLUMN id TO product_id;
```

```
[2026-04-24 15:01:13] financial_db> ALTER TABLE products RENAME COLUMN id TO product_id
[2026-04-24 15:01:13] completed in 63 ms
```

I ho validem:

```sql
SHOW COLUMNS FROM products;
```

Output | Result 14

6 rows

	Field	Type	Null	Key	Default	Extra
1	product_id	varchar(255)	YES		<null>	
2	product_name	varchar(255)	YES		<null>	
3	price	varchar(255)	YES		<null>	
4	colour	varchar(255)	YES		<null>	
5	weight	varchar(255)	YES		<null>	
6	warehouse_id	varchar(255)	YES		<null>	

Output | Result 14

```
[2026-04-24 15:02:18] financial_db> SHOW COLUMNS FROM products
[2026-04-24 15:02:18] 6 rows retrieved starting from 1 in 399 ms (execution: 17 ms, fetching: 382 ms)
```

Procedim a modificar els valors dels camps:

```sql
ALTER TABLE products
    MODIFY COLUMN product_id int NOT NULL AUTO_INCREMENT,
    MODIFY COLUMN product_name varchar(100) NOT NULL,
    MODIFY COLUMN price decimal(10,2) NOT NULL ,
    MODIFY COLUMN colour varchar(10) NOT NULL,
    MODIFY COLUMN weight decimal(10,2),
    MODIFY COLUMN warehouse_id varchar(10),
    ADD PRIMARY KEY (product_id);
```

```
[2026-04-24 15:04:39] financial_db> ALTER TABLE products
                                    MODIFY COLUMN product_id int NOT NULL AUTO_INCREMENT,
                                    MODIFY COLUMN product_name varchar(100) NOT NULL,
                                    MODIFY COLUMN price decimal(10,2) NOT NULL ,
                                    MODIFY COLUMN colour varchar(10) NOT NULL,
                                    MODIFY COLUMN weight decimal(10,2),
                                    MODIFY COLUMN warehouse_id varchar(10),
                                    ADD PRIMARY KEY (product_id)
[2026-04-24 15:04:40] 100 rows affected in 114 ms
```

I finalment, validem els canvis:

```sql
SHOW COLUMNS FROM products;
```

Output | Result 16

6 rows

	Field	Type	Null	Key	Default	Extra
1	product_id	int	NO	PRI	<null>	auto_increment
2	product_name	varchar(100)	NO		<null>	
3	price	decimal(10,2)	NO		<null>	
4	colour	varchar(10)	NO		<null>	
5	weight	decimal(10,2)	YES		<null>	
6	warehouse_id	varchar(10)	YES		<null>	

Output | Result 16

```
[2026-04-24 15:05:34] financial_db> SHOW COLUMNS FROM products
[2026-04-24 15:05:35] 6 rows retrieved starting from 1 in 389 ms (execution: 11 ms, fetching: 378 ms)
```

Ara procedim a crear la taula intermèdia:

```sql
CREATE TABLE IF NOT EXISTS operations_products
(
    operation_product_id int AUTO_INCREMENT,
    operation_id varchar(40),
    product_id int,
    PRIMARY KEY (operation_product_id)
);
```

```
[2026-04-24 16:22:55] financial_db> CREATE TABLE IF NOT EXISTS operations_products
                                    (
                                        operation_product_id int AUTO_INCREMENT,
                                        operation_id varchar(40),
                                        product_id int,
                                        PRIMARY KEY (operation_product_id)
                                    )
[2026-04-24 16:22:55] completed in 53 ms
```

Comprovem quants product_id hi ha com a màxim en algun dels registres de product_ids de la taula **operations**:

```sql
SELECT MAX(LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', '')) + 1) AS max_productes
FROM operations;
```

Output | max_productes:int

1 row

	max_productes
1	5

Com que hi ha com a màxim 5 product_id “junts”, executarem els següents procesos per inserir el operation_id i els respectius product_id. Com que son cinc procesos sobre 100000 registres, utilitzarem START TRANSACTION i COMMIT perquè s’executi de forma atòmica i evitar problemes si una de les queries dona algun error:

```
START TRANSACTION;
INSERT INTO operations_products (operation_id, product_id)
SELECT operation_id, product_id TRIM(SUBSTRING_INDEX(product_ids, ',', 1))
FROM operations
WHERE product_ids IS NOT NULL AND product_ids <> '';

INSERT INTO operations_products (operation_id, product_id)
SELECT operation_id, product_id TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 2), ',', -1))
FROM operations
WHERE (LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', ''))) >= 1;

INSERT INTO operations_products (operation_id, product_id)
SELECT operation_id, product_id TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 3), ',', -1))
FROM operations
WHERE (LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', ''))) >= 2;

INSERT INTO operations_products (operation_id, product_id)
SELECT operation_id, product_id TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 4), ',', -1))
FROM operations
WHERE (LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', ''))) >= 3;

INSERT INTO operations_products (operation_id, product_id)
SELECT operation_id, product_id TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 5), ',', -1))
FROM operations
WHERE (LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', ''))) >= 4;
COMMIT;
```

```
[2026-04-24 16:46:52] financial_db> START TRANSACTION
[2026-04-24 16:46:52] completed in 3 ms
[2026-04-24 16:46:52] financial_db> INSERT INTO operations_products (operation_id, product_id)
                                    SELECT operation_id, TRIM(SUBSTRING_INDEX(product_ids, ',', 1))
                                    FROM operations
                                    WHERE product_ids IS NOT NULL AND product_ids <> ''
[2026-04-24 16:46:54] 100,000 rows affected in 1 s 356 ms
[2026-04-24 16:46:54] financial_db> INSERT INTO operations_products (operation_id, product_id)
                                    SELECT operation_id, TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 2), ',', -1))
                                    FROM operations
                                    WHERE (LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', ''))) >= 1
[2026-04-24 16:46:55] 75,993 rows affected in 1 s 10 ms
[2026-04-24 16:46:55] financial_db> INSERT INTO operations_products (operation_id, product_id)
                                    SELECT operation_id, TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 3), ',', -1))
                                    FROM operations
                                    WHERE (LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', ''))) >= 2
[2026-04-24 16:46:56] 51,124 rows affected in 660 ms
[2026-04-24 16:46:56] financial_db> INSERT INTO operations_products (operation_id, product_id)
                                    SELECT operation_id, TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 4), ',', -1))
                                    FROM operations
                                    WHERE (LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', ''))) >= 3
[2026-04-24 16:46:56] 25,455 rows affected in 345 ms
[2026-04-24 16:46:56] financial_db> INSERT INTO operations_products (operation_id, product_id)
                                    SELECT operation_id, TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(product_ids, ',', 5), ',', -1))
                                    FROM operations
                                    WHERE (LENGTH(product_ids) - LENGTH(REPLACE(product_ids, ',', ''))) >= 4
[2026-04-24 16:46:56] 819 rows affected in 113 ms
[2026-04-24 16:46:56] financial_db> COMMIT
[2026-04-24 16:46:56] completed in 78 ms
```

Validem la càrrega:

```
SELECT *
FROM operations_products
ORDER BY operation_product_id DESC
LIMIT 100;
```

Output | financial_db.operations_products

100 rows | Tx: Auto | DDL

	operation_product_id	operation_id	product_id
1	361261	FFDB31C8-1CEF-4A47-9A0B-4942DB2786E4	31
2	361260	FF9A4013-EF03-4A36-A959-E2E677031F7E	16
3	361259	FF0F71B0-C6BB-4E05-B711-CCFF77A59574	72
4	361258	FEF564D0-4AD3-43FE-B43D-BC87979F279E	16
5	361257	FEABE7D9-E43A-47CD-9861-8DBA271F7180	5
6	361256	FE90375A-C64B-4FC2-A6D3-674ED2F673F0	16
7	361255	FE81BF14-C039-49F3-8329-8B28FD23889A	21
8	361254	FE60F938-9939-4078-A9C1-3ACDA1E474B7	16
9	361253	FE57F508-AF16-485D-9951-94FFAC3F3AC4	70

I finalment, assignarem les FK:

```
ALTER TABLE operations_products
    ADD CONSTRAINT fk_operations_products_operations
    FOREIGN KEY (operation_id) REFERENCES operations(operation_id);
```

```
[2026-04-24 17:01:36] financial_db> ALTER TABLE operations_products
                                    ADD CONSTRAINT fk_operations_products_operations
                                    FOREIGN KEY (operation_id) REFERENCES operations(operation_id)
[2026-04-24 17:01:39] 253,391 rows affected in 3 s 401 ms
```

```
ALTER TABLE operations_products
    ADD CONSTRAINT fk_operations_products_products
    FOREIGN KEY (product_id) REFERENCES products(product_id);
```

```
[2026-04-24 17:01:44] financial_db> ALTER TABLE operations_products
                                    ADD CONSTRAINT fk_operations_products_products
                                    FOREIGN KEY (product_id) REFERENCES products(product_id)
[2026-04-24 17:01:47] 253,391 rows affected in 3 s 135 ms
```

Validem els canvis:

```
SHOW COLUMNS FROM operations_products;
```

Output | Result 31

3 rows

	Field	Type	Null	Key	Default	Extra
1	operation_product_id	int	NO	PRI	<null>	auto_increment
2	operation_id	varchar(40)	YES	MUL	<null>	
3	product_id	int	YES	MUL	<null>	

Output | Result 31

```
[2026-04-24 17:04:12] financial_db> SHOW COLUMNS FROM operations_products
[2026-04-24 17:04:12] 3 rows retrieved starting from 1 in 388 ms (execution: 12 ms, fetching: 376 ms)
```

Amb això, hem acabat de crear la taula intermèdia.

Com a punt final, s'hauria d'eliminar el camp product_ids de la taula **operations**, ja que si ara mantenim aquest camp, pot provocar en els millors dels casos redundància, però pot també provocar conflictes quan s'insereixin més operacions, ja que pot haver-hi dues realitats, una la que indiqui aquest camp i una altre la que indiqui la taula **operations_products**.

A més, eliminar aquest camp estalvia espai en disc i memòria RAM al realitzar consultes.

```
ALTER TABLE operations
DROP COLUMN product_ids;
```

```
[2026-04-24 17:22:23] financial_db> ALTER TABLE operations
                                    DROP COLUMN product_ids
[2026-04-24 17:22:23] completed in 156 ms
```

Validem el canvi:

```
SHOW COLUMNS FROM operations;
```

Output | Result 33

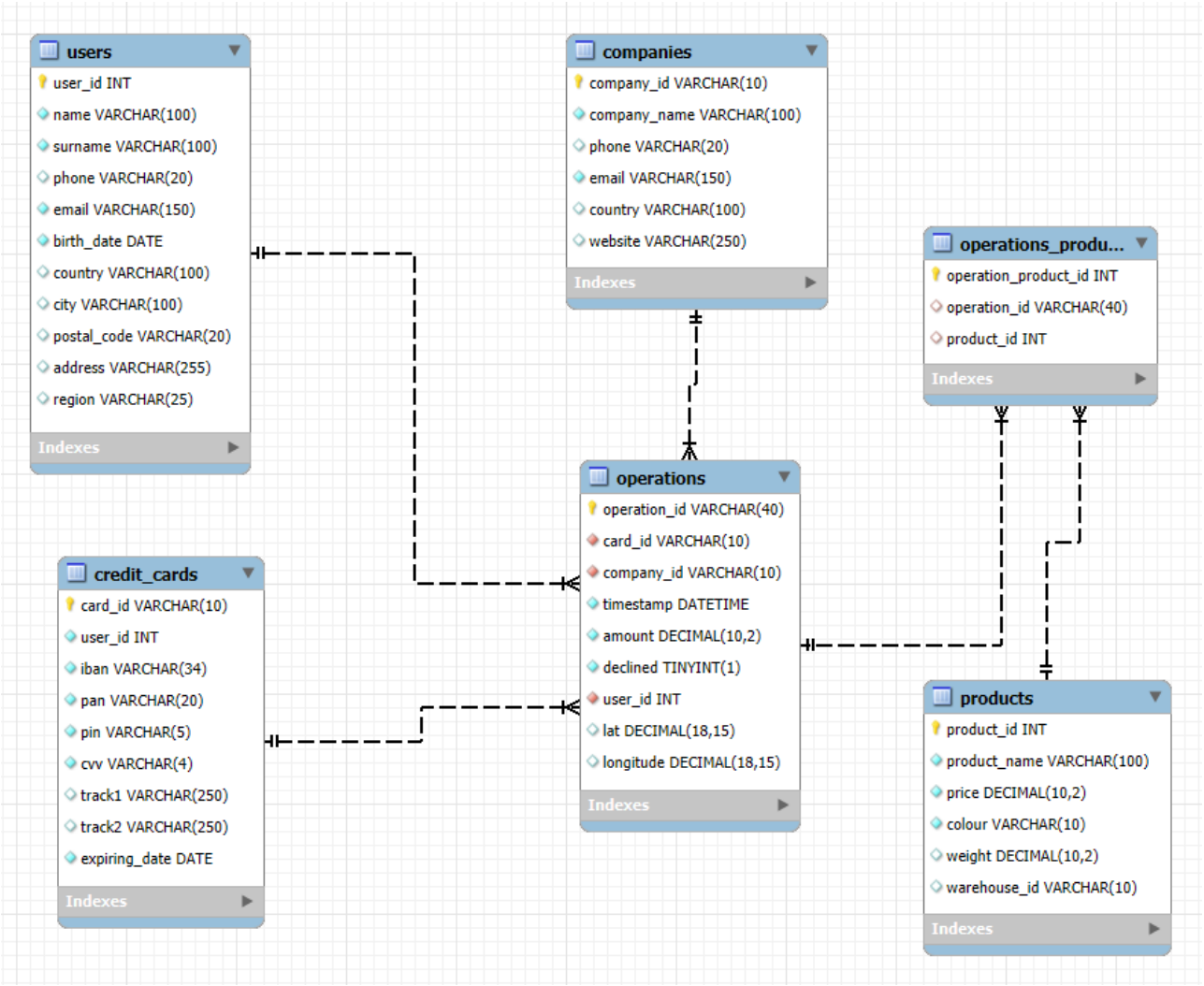
9 rows

	Field	Type	Null	Key	Default	Extra
1	operation_id	varchar(40)	NO	PRI	<null>	
2	card_id	varchar(10)	NO	MUL	<null>	
3	company_id	varchar(10)	NO	MUL	<null>	
4	timestamp	datetime	NO		<null>	
5	amount	decimal(10,2)	NO		<null>	
6	declined	tinyint(1)	NO		<null>	
7	user_id	int	NO	MUL	<null>	
8	lat	decimal(18,15)	YES		<null>	
9	longitude	decimal(18,15)	YES		<null>	

Output | Result 33

```
[2026-04-24 17:23:31] financial_db> SHOW COLUMNS FROM operations
[2026-04-24 17:23:31] 9 rows retrieved starting from 1 in 374 ms (execution: 12 ms, fetching: 362 ms)
```

Amb tots aquests canvis, ara financial_db té un model de floquet de neu, amb una taula intermèdia entre **operations** i **products**:



Genera la següent consulta:

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

Seleccionem el id i el nom de producte per evitar possibles confusions en el cas que el nom estigui duplicat. Calculem amb COUNT el total de vegades que s'ha venut cada producte. Agrupem per l'id i el nom del producte i ordenem pel total de vendes.

```sql
SELECT p.product_id AS ID_Producte, p.product_name AS Nom_Producte, COUNT(op.product_id) AS Total_Vendes
FROM products AS p
JOIN operations_products AS op ON p.product_id = op.product_id
GROUP BY p.product_id, p.product_name
ORDER BY Total_Vendes DESC;
```

Output | Necessitem conèix... venut cada producte.

100 rows

	ID_Producte	Nom_Producte	Total_Vendes
1	52	riverlands the duel	2654
2	29	Tully maester Tarly	2635
3	21	duel Direwolf	2609
4	16	the duel warden	2608
5	66	mustafar jinn	2601
6	87	sith Jade	2598
7	33	duel warden	2597
8	48	rock Renly in	2597
9	23	riverlands north	2593
10	68	Stark Karstark	2589

Output | Necessitem conèix... venut cada producte.

```
[2026-04-24 17:39:40] financial_db> SELECT p.product_id AS ID_Producte, p.product_name AS Nom_Producte, COUNT(op.product_id) AS Total_Vendes
                                    FROM products AS p
                                    JOIN operations_products AS op ON p.product_id = op.product_id
                                    GROUP BY p.product_id, p.product_name
                                    ORDER BY Total_Vendes DESC
[2026-04-24 17:39:41] 100 rows retrieved starting from 1 in 580 ms (execution: 211 ms, fetching: 369 ms)
```